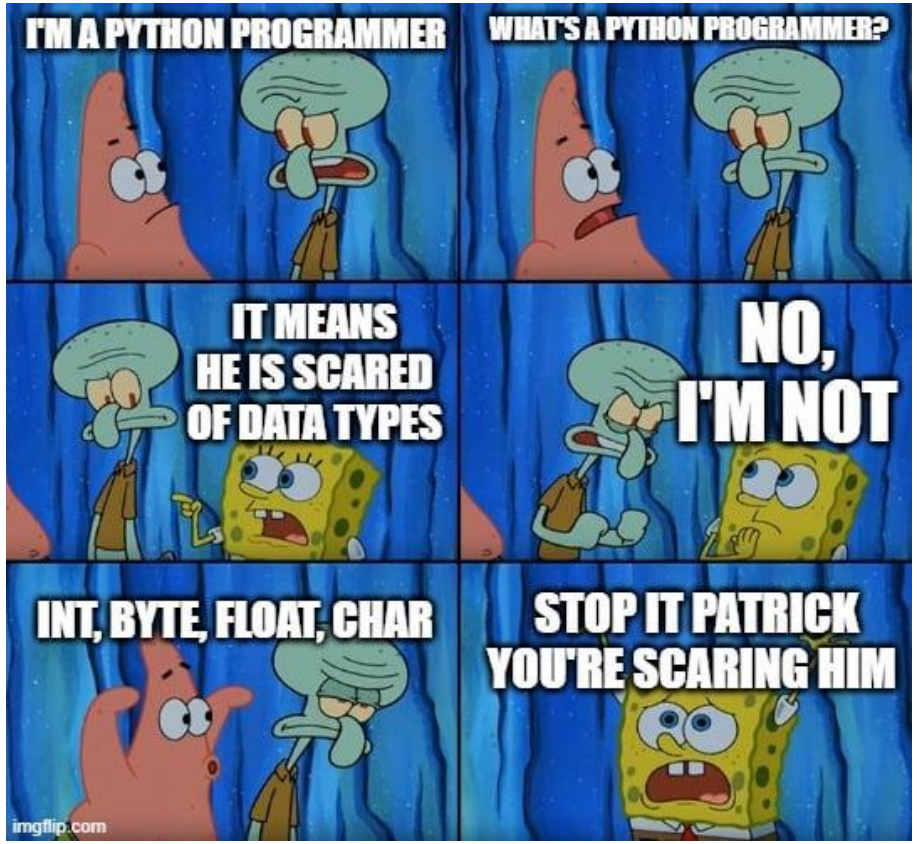
1. Basis data types

1A.1-basis_data_types.ipynb

KU LEUVEN

Basis data types in Python

- Integers
- Floats
- Complexe nummers
- Booleans
- Strings



https://www.reddit.com/r/ProgrammerHumor/comments/ns4met/im_a_python_programmer/

Integers

- Gehele getallen
- Worden automatisch herkend door Python
- In Python: van de klasse/het type `int`

De ingebouwde functie `type()` geeft het type/de klasse van de variabele

Code:

```
# integers
a = 5

print(a)
print(type(a))
```

Output:

```
5
<class 'int'>
```


KU LEUVEN

Floating-point nummers (floats)

- Kommagetallen
- Kunnen in de wetenschappelijke notatie gebruikt worden
- In Python: van het type `float`

Code:

```
b = 5.3
c = 2.21e-10
d = 2.21e10

print(b)
print(c)
print(type(b))
print(type(c))
print(int(d))
```

Output:

```
5.3
2.21e-10
<class 'float'>
<class 'float'>
22100000000
```

Merk op dat dit een conversie is – hier komen we later op terug

KU LEUVEN

Handige ingebouwde functies voor cijfers

Code:

```
a = 2
b = 2.0202
print(round(a, 2))      # rond af tot 2 cijfers na komma
print(round(b, 2))
print(max(a, b))        # werkt ook op lijsten en sets
print(min(a, b))        # idem
print(pow(a, 10))       # machtsverheffing
```

Output:

```
2
2.02
2.0202
2
1024
```


KU LEUVEN

Opgelet: precisie van floats

- Floats worden binair benaderd: kleine afrondingsfouten zijn normaal
- Vergelijk floats dus beter niet met ==, maar met een kleine marge
- `round()` is voor weergave, niet voor exacte berekeningen (maar voldoende voor dit vak)

Code:

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)

# afronden voor de vergelijking
print(round(0.1 + 0.2, 2) == 0.3)

# of werken met een marge
print(abs((0.1 + 0.2) - 0.3) < 0.000001)
```

Output:

```
0.30000000000000004
False


True


True
```

Booleans

- Waar/vals waarden (`True`/`False` zijn gereserveerde keywords)
- In Python: van het type `bool`

Code:

```
# booleans
waar = True
vals = False

print(waar)
print(type(waar))
```

Output:

```
True
<class 'bool'>
```


KU LEUVEN

Strings

- Sequentie van tekens/characters
- Sterke overeenkomsten met lijsten (zie later) – beiden zijn itereerbare types:
  - Makkelijk te gebruiken/manipuleren vanwege het gebruik van indices
  - **Let wel:** strings kunnen niet veranderen zoals lijsten
- Kunnen aangemaakt worden met zowel enkele als dubbele aanhalingstekens (‘ en “ maar niet door elkaar):
  - ‘Escape characters’ zoals ‘\’ nodig om haakjes tussen de aanhalingstekens te gebruiken:

Code:

```
kleine_zin = 'Ik ben daarstraks \'naar de les geweest\''
print(kleine_zin)
```

Output:

```
Ik ben daarstraks 'naar de les geweest'
```


KU LEUVEN

Strings

Code:

```
# strings
ch1 = 'b'
ch2 = "b"
str1 = 'blabla'

print(ch1)
print(ch2)
print(str1)
print(type(ch1))
print(type(ch2))
print(type(str1))
```

Er is geen onderscheid in type tussen enkele karakters/tekens en strings (<> Java, C++, etc.)

Output:

```
b
b
blabla
<class 'str'>
<class 'str'>
<class 'str'>
```

Concatenatie van strings

- Er bestaan verschillende operatoren in Python om bewerkingen uit te voeren zoals +,-,/,*,**, etc.
  - We gaan deze later nog bespreken
- Weet nu: strings kunnen aan elkaar geplakt (geconcateneerd) worden met ‘+’

+= combineert concateneren met toewijzen (werkt ook bij getallen)

Code:

```
string1 = 'eerste'
string2 = 'tweede'
concatenatie = string1 + ' ' + string2
print(concatenatie)

string1 += ' en nog wat erachter'
print(string1)
```

Output:

```
eerste tweede

eerste en nog wat erachter
```

Nuttige tekens in strings

- ‘\n’ zorgt voor een nieuwe lijn
- ‘\t’ voegt een tab in

Code:

```
zin_over_meerdere_lijnen = 'Ik ben daarstraks gaan:\n\tfietsen\n\tzwemmen'
print(zin_over_meerdere_lijnen)
```

Output:

```
Ik ben daarstraks gaan:
        fietsen
        zwemmen
```


KU LEUVEN

Conversies tussen types

- Ingebouwde functies (eigenlijk constructors, dit zien we later nog uitgebreid) die het toelaten om types om te zetten

Code:

```
a = 'True'
print('a:', type(a))
a2 = bool(a)
print('a2:', type(a2))

a = '5'
print('a:', type(a))
a2 = int(a)
print('a2:', type(a2))
print('a:', a, 'a2:', str(a2))
```

Output:

```
a: <class 'str'>

a2: <class 'bool'>


a: <class 'str'>

a2: <class 'int'>
a: 5 a2: 5
```

!Let op! Je kan natuurlijk niet zomaar 'five' omzetten naar een integer of 'waar' naar 'True'

`bool()` test enkel of het object een waarde heeft

KU LEUVEN

Oefeningen 1.1: voorspel de output

```
a = 7
b = 2
print(type(a / b), type(a // b))

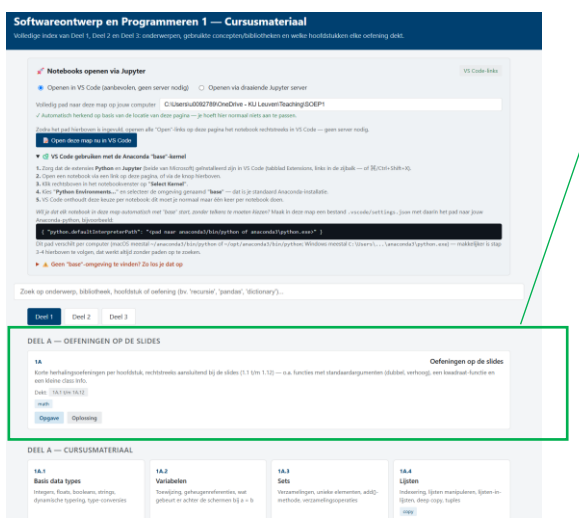
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)

s = "Leuven"
print(s[0], s[-1], len(s))
print("a\tb\nc")
```

1. **Voorspel eerst (30 s)**
2. **Dan runnen en verbeteren (zie notebook)**

Opzet van deze oefeningen:

- Ze zijn heel kort
- Helpen om de syntax in te oefenen
- We reproduceren eerst wat uit de slides
- Daarna verbeteren we de code in de notebook zodat de asserts uitkomen
  - Deze kan je vinden in de (online) cursus



https://johannesdesmedt.github.io/softwareontwerp_en_programmeren/

2. Variabelen

1A.2-variabelen.ipynb

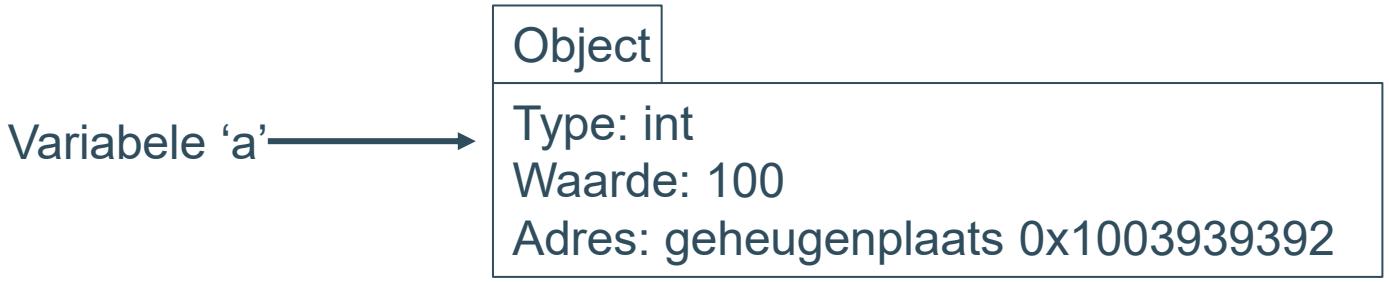




https://programmerhumor.io/programming-memes/type-stronk/

Variabele aanmaken

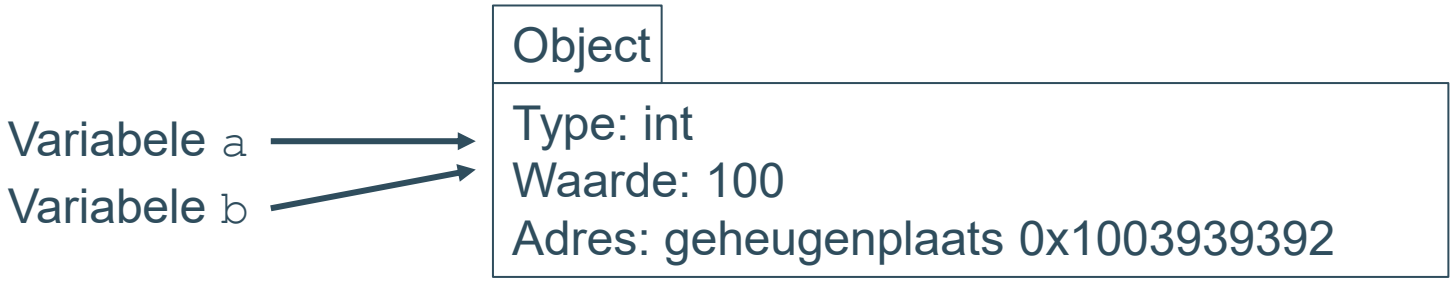
- Zoals in de vorige slides gebruikt, dient het ‘=‘ symbool om een variabele een waarde toe te wijzen van een bepaald type
- Wat gebeurt er bij de toewijzing?
  - Er wordt een verwijzing gemaakt naar een (nieuw) object
  - Bv. ‘a = 100’: een verwijzing wordt gemaakt naar (de geheugenplaats van) een nieuw object van het *type* `int` met *waarde 100*



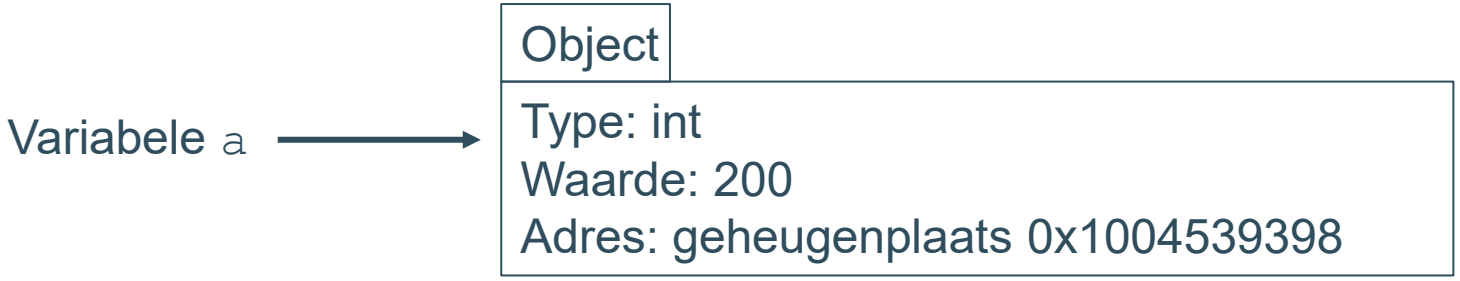
KU LEUVEN

Variabele aanmaken

- Er wordt een verwijzing gemaakt naar een object:
  - `a = b = 100` : zowel `a` als `b` verwijzen naar hetzelfde object



  - `a = 200` : nieuwe verwijzing naar een nieuw `int` object



KU LEUVEN

Variabele wijzigen

- Dit werkt met simpele objecten (basis types) zoals integers en strings, maar niet met complexere objecten zoals lijsten

Code:

```
a = b = 'johannes'
print('a:', a)
print('b:', b)

a += ' de smedt'
print('a - nu:', a)
print('b - nu:', b)
```

Output:

```
a: johannes
b: johannes

a - nu: johannes de smedt
b - nu: johannes
```

- Er wordt een nieuw string object aangemaakt wanneer `a` gewijzigd wordt

Code:

```
a = b = [1, 2, 3]
print('a:', a)
print('b:', b)

a.pop() # verwijder laatste element
print('a - nu:', a)
print('b - nu:', b)
```

Output:

```
a: [1, 2, 3]
b: [1, 2, 3]

a - nu: [1, 2]
b - nu: [1, 2]
```

- Nu wordt er geen nieuw object aangemaakt en `a` en `b` blijven naar hetzelfde object verwijzen
- Voor niet-basis objecten moeten we dan een deep copy maken (zie later)

Onthoud: er is altijd een deep copy nodig behalve voor getallen en strings (basis types), daar wordt bij elke wijziging een nieuw object aangemaakt

KU LEUVEN

Variabele wijzigen (cont.)

- Waarom wordt er automatisch een nieuw object gemaakt van de string maar niet van de lijst?
  - String/int/float/boolean verwijzen naar object met 1 waarde:
    - Kopiëren van waarde is vanzelfsprekend bij een wijziging van de waarde
  - Samengestelde types (lijst/set/eigen gemaakt object) bevatten (mogelijks) meerdere waarden:
    - Geen eenduidig overzicht welke waarden moeten worden behouden/aangepast

Opgelet

- Wederom, dynamische typering is gevaarlijk:

Code:

```
a = 4
b = 5
print(a + b)
a = 'vier'
print(a + b)
```

Na het veranderen van `a` in een string kunnen we `+` niet meer gebruiken omdat we 2 verschillende types 'optellen'
➔ + bij strings: concatenatie
➔ + bij integers: optellen

Output:

```
9
Traceback (most recent call last):
  File "~/SOEP1/1-basis_data_types.py", line 51, in <module>
    print(a + b)
TypeError: can only concatenate str (not "int") to str
```

Code:

```
a = 4
b = 5
print(a + b)
a = 'vier'

b = 'vijf'
print(a + b)
```

We kunnen wel 2 strings concateneren wanneer `b` ook een string is geworden

Output:

```
9
viervijf
```


KU LEUVEN

In Java

Code:

```
package be.kuleuven.soep1;

public class Variabelen {

	public static void main(String[] args) {
		int a = 5;


		System.out.println(a);
		a = "Johannes”;
	}
}
```

Output:

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
Type mismatch: cannot convert from String to int

at soep1/be.kuleuven.soep1.Variabelen.main(Variabelen.java:10)
```

Deze code kan zelfs niet gestart worden door de fout

Oefening 1.2: voorspel de output

```
a = b = 100
a = a + 1
print(a, b)

x = [1, 2]
y = x
y.append(3)
print(x, y)
```

1. **Voorspel eerst (30 s)**
2. **Dan runnen en verbeteren**

Hint: je maakt een copy met `list()` of `copy()` – zoek de details desnoods online

3. Composiete types

KU LEUVEN

Composiete data types

- Verzamelingen (sets)
- Lijsten (lists/tuples)
- Woordenboeken (dictionaries)
- Zelfgemaakte klasses (komen iets later, 1.10 (notebook 1.13))

KU LEUVEN

Verzamelingen

1A.3-sets.ipynb

KU LEUVEN

Introductie

- Zijn ongeordende collecties (maar wel itereerbaar)
- Alle elementen zijn uniek (maar niet noodzakelijk van hetzelfde type):
  - Mogen niet ‘veranderbaar zijn’, dus bv. geen andere sets of lijsten
    - Moeten kunnen gecontroleerd worden op hun uniekheid (en lijsten kunnen veranderd worden zodat ze hetzelfde worden)
- Makkelijk om vergelijkingen mee te maken via deelverzamelingen, etc.

Toevoegen elementen met `.add()`

Code:

```
kleine_set = set() # lege set
print(kleine_set)

kleine_set.add(1)
kleine_set.add('johannes')
kleine_set.add(1)
kleine_set.add(2)

print(kleine_set)
```

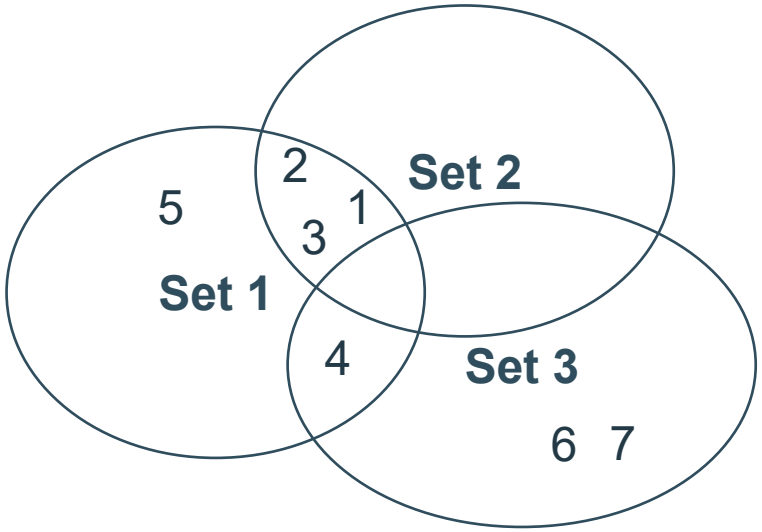
:Output

```
set()

{'johannes', 1, 2}
```


KU LEUVEN

Elementen



Aanmaken kan dus met {}, maar er moeten telkens al elementen meegegeven worden – anders wordt het object aanzien als een dictionary (zie later)

Code:

```
# check of iets in een set zit
nieuwe_set = {1, 2, 3, 5} # nog een manier om een set aan te maken
print(1 in nieuwe_set)
print(4 in nieuwe_set)

set_1 = {1, 2, 3, 4, 5}
set_2 = {1, 2, 3}
set_3 = {4, 6, 7}
print(set_1.union(set_3))
print(set_1.difference(set_2))
print(set_1.intersection(set_2))
print(set_2.issubset(set_1))
```

De ∈ operator

Output:

```
True
False

{1, 2, 3, 4, 5, 6, 7}
{4, 5}
{1, 2, 3}
True
```


KU LEUVEN

Methodes vs. operatoren

- Objecten hebben gereserveerde functies die op hen kunnen toegepast worden – methodes genaamd (oproepen via . – zie later)
- In plaats van methodes (van het object) te gebruiken, kunnen ook operatoren gebruikt worden (over het object/de objecten):

Code:

```
set_1 = {1, 3, 4}
set_2 = {1, 2}
set_3 = {4, 6}
print(set_1 | set_2 | set_3, set_1.union(set_2).union(set_3))
print(set_1 & set_2, set_1.intersection(set_2))
print(set_1 - set_2, set_1.difference(set_2))
```

Output:

```
{1, 2, 3, 4, 6} {1, 2, 3, 4, 6}
{1} {1}
{3, 4} {3, 4}
```


KU LEUVEN

Verwijderen en toevoegen

- Elementen toevoegen en verwijderen kan via deze methodes:
  - `add()`: toevoegen van 1 of meerdere elementen
  - `update()`: toevoegen van 1 of meerdere elementen uit een itereerbaar object
  - `remove()`: verwijdert een element maar geeft een foutenboodschap als het element niet aanwezig is
  - `discard()`: probeert een element te verwijderen, of het aanwezig is of niet
  - `pop()`: verwijdert een willekeurig element (foutenboodschap bij lege set)
  - `clear()`: maakt de set leeg

Voorbeeld

Dit element is niet meer aanwezig, maar `.discard()` geeft hier geen foutenboodschap voor

Code:

```
set_4 = {1, 4, 'Johannes'}
set_4.add('Johanna')
print(set_4)
set_4.pop()
print(set_4)
set_4.discard('Johannes')
print(set_4)
set_4.discard('Johannes')
print(set_4)
set_4.remove('Johannes')
```

Output:

```
{1, 4, 'Johanna', 'Johannes'}

{4, 'Johanna', 'Johannes'}

{4, 'Johanna'}

{4, 'Johanna'}

Traceback (most recent call last):
  File "/Users/johannesdesmedt/Desktop/Teaching/SOEP1/Deel 1/1.3-sets.py", line 60, in <module>
    set_4.remove('Johannes')  # dit geeft een foutmelding
KeyError: 'Johannes'
```

Dit element is niet aanwezig

KU LEUVEN

Lijsten

1A.4-lijsten.ipynb

KU LEUVEN

Introductie

- Een collectie van eender welk type objecten opgeslagen in de volgorde waarin ze worden meegegeven
- Volledig geïndexeerd en makkelijk te navigeren

Code:

```
lijst_1 = [1, 2, 'johannes', 2]
print(lijst_1)
print(lijst_1[0])
print(lijst_1[1])

lijst_2 = list()
lijst_2.append(2)
print(lijst_2)
```

Output:

```
[1, 2, 'johannes', 2]
1
2


[2]
```

Zoals in de meeste programmeertalen tellen we vanaf 0

Indexering

Code:

```
lijst_3 = ['a', 'b', 'c', 'f', 1, 3, 4]
print(lijst_3)
print(lijst_3[0])
print(lijst_3[3])
print(lijst_3[-1])
print(lijst_3[1:3])
print(lijst_3[1:5:2])
print(lijst_3[-3:])
print(lijst_3[-4:-2])
print(lijst_3[::-1])
```

Output:

```
['a', 'b', 'c', 'f', 1, 3, 4]
a
f
4
['b', 'c']
['b', 'f']
[1, 3, 4]
['f', 1]
[4, 3, 1, 'f', 'c', 'b', 'a']
```

Volledige lijst:
Eerste element:
Vierde element:
Laatste element:
Elementen 2 **tot** 4:
Elementen 2, 4 (2 **tot** 6 per 2):
Laatste 3 elementen:
4de laatste **tot** 2de laatste elementen:
Omgekeerde lijst:

Strings zijn ook itereerbare objecten die dezelfde indexering hanteren

KU LEUVEN

Verwijderen en toevoegen

Als we de + operator gebruiken moeten we een andere lijst aan onze lijst koppelen (hier dus met haken om een lijst aan te maken met 1 element). De + operator werkt zoals `extend()`.

Code:

```
lijst_4 = [1, 2, 'johannes']
print(lijst_4)
lijst_4.append('Johanna')
print(lijst_4)
lijst_4.append([2, 'Johannes'])
print(lijst_4)
lijst_4.extend([3, 5, 6])
print(lijst_4)
lijst_4 = lijst_4 + [0]
print(lijst_4)
lijst_4.pop()
print(lijst_4)
lijst_4.insert(3, 'blabla')
print(lijst_4)
del lijst_4[2:8]
print(lijst_4)
lijst_4.remove('Johannes')
print(lijst_4)
```

Output:

```
[1, 2, 'johannes']

[1, 2, 'johannes', 'Johanna']

[1, 2, 'johannes', 'Johanna', [2, 'Johannes'] ] # dit is een lijst in een lijst

[1, 2, 'johannes', 'Johanna', [2, 'Johannes'], 3, 5, 6]

[1, 2, 'johannes', 'Johanna', [2, 'Johannes'], 3, 5, 6, 0]

[1, 2, 'johannes', 'Johanna', [2, 'Johannes'], 3, 5, 6]

[1, 2, 'johannes', 'blabla', 'Johanna', [2, 'Johannes'], 3, 5, 6]

[1, 2, 6]

Traceback (most recent call last):  File ~/SOEP1/1-lijsten.py", line 42, in
<module>  lijst_4.remove('Johannes') ValueError: list.remove(x): x not in list
```


KU LEUVEN

Handige methodes

- `count(el)`: telt het aantal voorkomens van element `el`
- `index(el)`: geeft de index terug van element `el`, of een foutenmelding

Code:

```
lijst_5 = [2, 2, 1, 'johannes', 'Johanna']
print('Index van eerste 2:', lijst_5.index(2))
print('Aantal keer dat 2 voorkomt:', lijst_5.count(2))
print("Aantal keer dat 'blabla' voorkomt:", lijst_5.count('blabla'))
try:
    lijst_5.index('blabla') # dit geeft een foutmelding
except ValueError as e:
    print('Fout:', e)
    print("'blabla' is niet aanwezig in de lijst")
```

Output:

```
Index van eerste 2: 0
Aantal keer dat 2 voorkomt: 2
Aantal keer dat 'blabla' voorkomt: 0
Fout: 'blabla' is not in list 'blabla' is niet aanwezig in de lijst
```


KU LEUVEN

Nesten van lijsten

- Eender welk soort object kan in een lijst gestopt worden:
  - Dus ook andere lijsten
- Zo kunnen matrices worden gebouwd:

Code:

```
hoofdlijst = list()
sublijst_1 = [1, 2, 3]
sublijst_2 = ['johannes', 5, 6]
hoofdlijst.extend([sublijst_1, sublijst_2])
print(hoofdlijst)
print(hoofdlijst[1][0])
print(hoofdlijst[0][2])
```

Output:

```
[[1, 2, 3], ['johannes', 5, 6]]
Johannes                (tweede rij, eerste kolom)
3                       (eerste rij, derde kolom)
```


KU LEUVEN

Shallow copy vs. deep copy

Code:

```
import copy

eerste_lijst = [1, 2, [3, 4]]
tweede_lijst = list(eerste_lijst) # shallow copy

tweede_lijst.append('extra')
print(eerste_lijst)
print(tweede_lijst)

tweede_lijst[2][0] = 5
print(eerste_lijst)
print(tweede_lijst)

derde_lijst = copy.deepcopy(eerste_lijst)
derde_lijst[2][0] = 1
print(eerste_lijst)
print(derde_lijst)
```

Output:

```
[1, 2, [3, 4]]
[1, 2, [3, 4], 'extra']

[1, 2, [5, 4]]
[1, 2, [5, 4], 'extra']

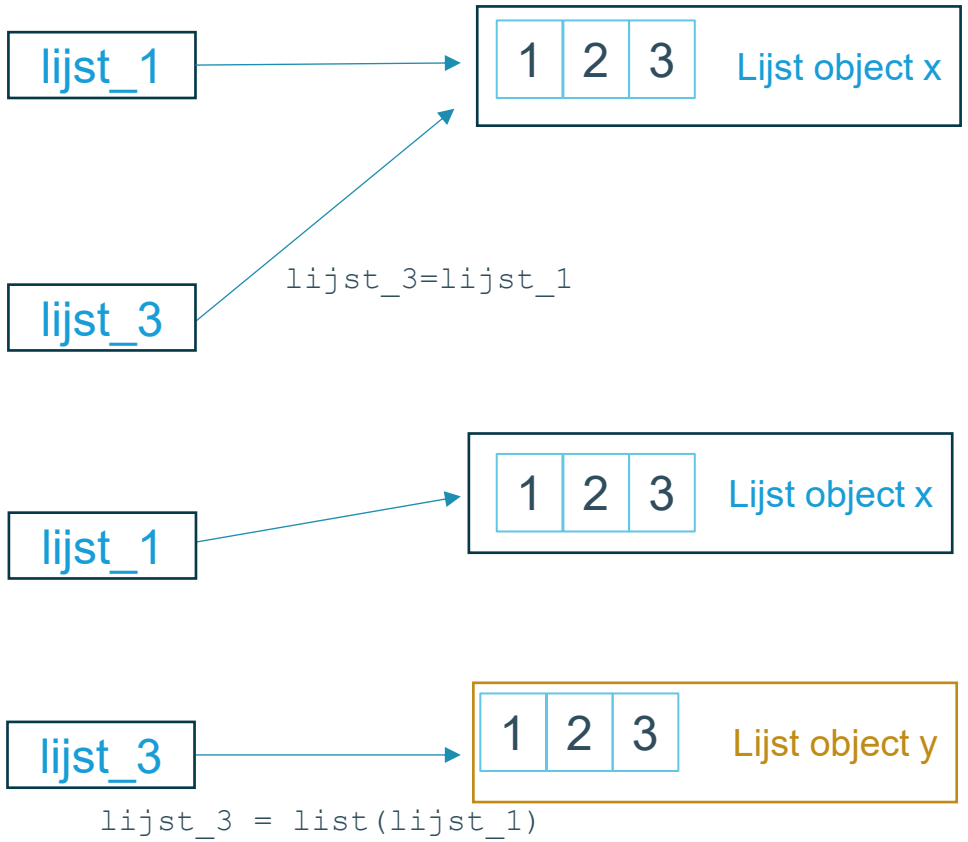
[1, 2, [5, 4]]
[1, 2, [1, 4]]
```

eerste_lijst[2][0]

Een ‘shallow’ kopie van een lijst kan gemaakt worden met `list()`, maar deze zal niet de onderliggende elementen kopiëren – de lijst in de lijst wordt gewoon in beide lijsten gewijzigd (3 -> 5)

Om dit te vermijden kan een `deepcopy()` gebruikt worden

Extra voorbeeld




```
lijst_1 = [1,2,3]
lijst_3 = lijst_1

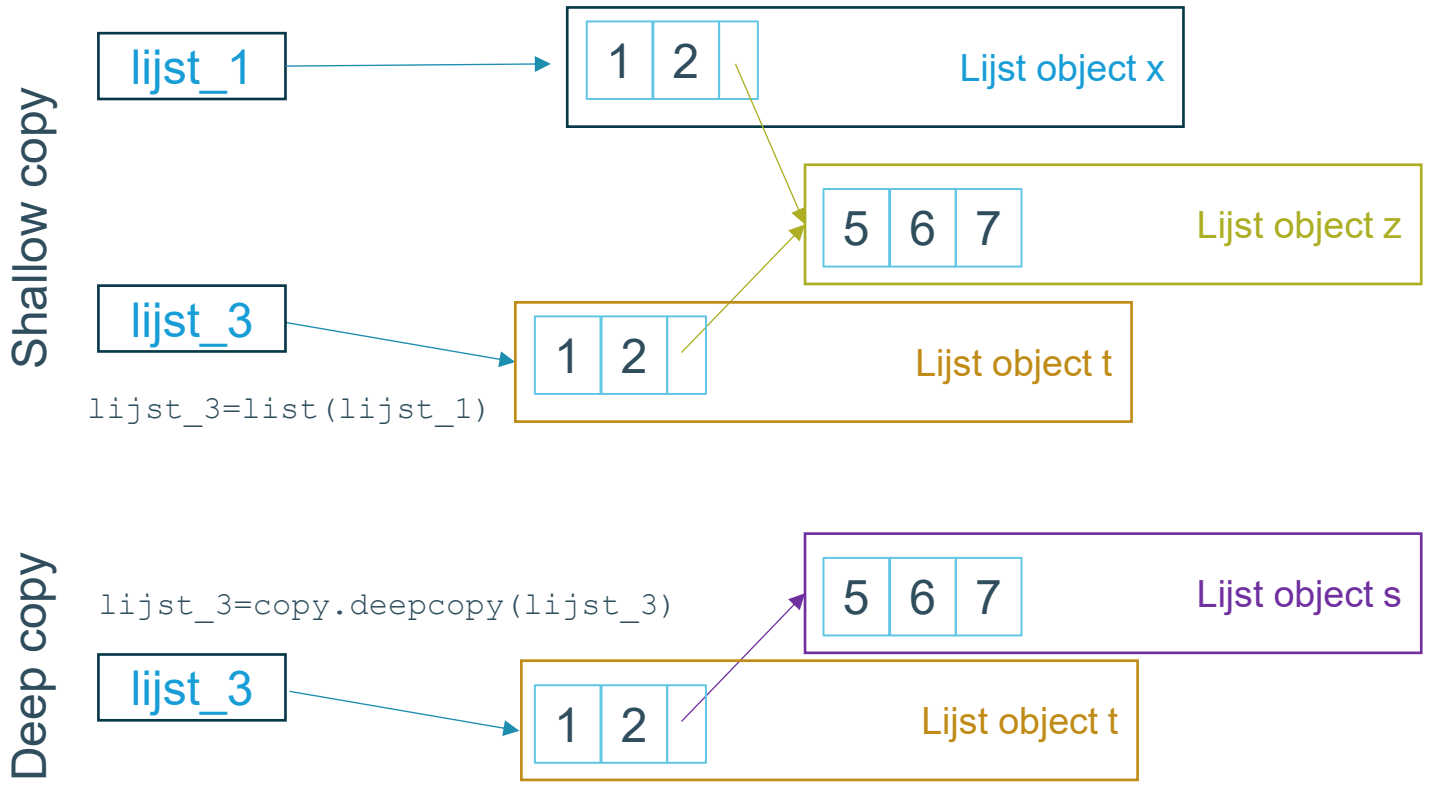

# beide lijsten verwijzen naar dezelfde plaats in
het geheugen
print(lijst_1, lijst_3)
lijst_3[2] = 100
print(lijst_1, lijst_3)


lijst_1 = [1,2,3]
# nu wordt een nieuw object op een nieuwe plaats
aangemaakt
# voor lijst_3
lijst_3 = list(lijst_1)


print(lijst_1, lijst_3)
lijst_3[2] = 100
print(lijst_1, lijst_3)
```


KU LEUVEN

Extra voorbeeld grafisch (2)




```
# nu bevat de lijst een andere lijst
lijst_1 = [1,2, [5,6,7]]
lijst_3 = lijst_1


print(lijst_1, lijst_3)
lijst_3[2][1] = 100
print(lijst_1, lijst_3)


lijst_1 = [1,2, [5,6,7]]
# we maken een shallow copy
lijst_3 = list(lijst_1)


print(lijst_1, lijst_3)
lijst_3[2][1] = 100
# het probleem blijft bestaan want de lijst in de lijst
verwijst nog altijd naar dezelfde geheugenplaats
print(lijst_1, lijst_3)


import copy
lijst_1 = [1,2, [5,6,7]]
lijst_3 = copy.deepcopy(lijst_1)


# na een deep copy zal elk element (ook een samengesteld)
# een nieuwe plaats in het geheugen krijgen
print(lijst_1, lijst_3)
lijst_3[2][1] = 100
print(lijst_1, lijst_3)
```


KU LEUVEN

Tuples

- Gelijkaardig aan lijsten, maar niet-veranderbaar (immutable)
- Kunnen dus bv. wel in sets gebruikt worden als hun elementen ook immutable zijn
- Gebruiken dezelfde indexering als lijsten
- Gebruiken ronde haakjes bij het aanmaken en komen vaak voor om bv. paren te gebruiken (bv. Key-value pairs in dictionaries – zie later)

Code:

```
paar = (1, [2, 3, 'Johannes'])
print(paar)
print(paar[0])
paar.append('3')
```

Toevoegen gaat uiteraard niet want ze zijn niet-veranderbaar

Output:

```
(1, [2, 3, 'Johannes'])
1
Traceback (most recent call last):
  File "~/SOEP1/1-lijsten.py", line 60, in <module>
    paar.append('3')
AttributeError: 'tuple' object has no attribute 'append'
```


KU LEUVEN

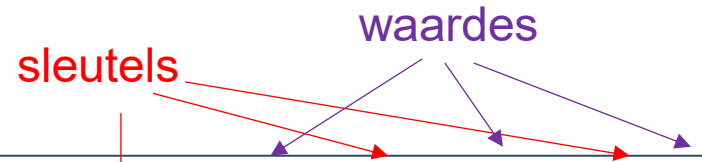
Dictionaries

1A.5-dictionaries.ipynb

KU LEUVEN

Introductie

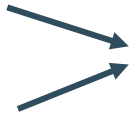
- Een collectie van sleutel-waarde (key-value) paren zoals misschien bekend uit databases:
  - Sleutels zijn uniek, waardes kunnen uiteraard meerdere keren voorkomen
  - Sleutels moeten niet van hetzelfde type zijn (maar wel immutable – zie sets)



Code:

```
woordenboek = {'Johannes': 'SOEP1', 'Jan': 'SOEP2', 2: 'SOEP3'}
print(woordenboek)

woordenboek _2 = dict()
print(woordenboek _2)
woordenboek _2['Johannes'] = 'SOEP1'
print(woordenboek _2)
woordenboek_2['Johannes'] = 'SOEP2'
print(woordenboek _2)
```



Waarde wordt overschreven

Output:

```
{'Johannes': 'SOEP1', 'Jan': 'SOEP2', 2: 'SOEP3'}

{}

{'Johannes': 'SOEP1'}

{'Johannes': 'SOEP2'}
```


KU LEUVEN

Opvragen van data: methodes

- `.items()` geeft de data als paren terug
- `.keys()`/`.values()` geven de sleutels respectievelijk de waarden terug
- Let wel: `dict_items`/`dict_keys`/`dict_values` objecten worden teruggegeven - deze kunnen omgezet worden in bv. een set of lijst om makkelijk behandeld te worden (of itereer zelf – zie volgende slide)

Code:

```
dict_1 = {'Johannes': 'SOEP1', 'Johanna': 'SOEP2'}
print(dict_1.items())
print(dict_1.keys())
print(type(dict_1.keys()))
print(list(dict_1.keys()))
print(dict_1.values())
print(list(dict_1.values())[0])
```

Output:

```
dict_items([('Johannes', 'SOEP1'), ('Johanna', 'SOEP2')])
dict_keys(['Johannes', 'Johanna'])
<class 'dict_keys'>
['Johannes', 'Johanna']
dict_values(['SOEP1', 'SOEP2'])
SOEP1
```

Opvragen van data

Zelf itereren:

- Merk op dat we telkens een paar terugkrijgen dat we in 2 variabelen stoppen, `key` en `value`

Code:

```
# zelf itereren
for key, value in dict_1.items():
    print('Sleutel:', key, '\twaarde:', value)
```

Output:

```
Sleutel: Johannes    waarde: SOEP1
Sleutel: Johanna     waarde: SOEP2
```


KU LEUVEN

Verwijderen en samenvoegen

Code:

```
dict_1 = {'Belgie': 10, 'Nederland': -10}
dict_2 = {'Frankrijk': 2, 'Duitsland': 2}
print(dict_1)
dict_1.update(dict_2)
print(dict_1)

del dict_1['Belgie']
print(dict_1)

# pop verwijdert de sleutel en geeft de waarde als
resultaat
waarde = dict_1.pop('Nederland')
print(dict_1)
print(waarde)
```

Output:

```
{'Belgie': 10, 'Nederland': -10}

{'Belgie': 10, 'Nederland': -10, 'Frankrijk': 2, 'Duitsland': 2}


{'Nederland': -10, 'Frankrijk': 2, 'Duitsland': 2}




{'Frankrijk': 2, 'Duitsland': 2}
-10
```


KU LEUVEN

Veilig opvragen: `get()`

- `get(sleutel)`: geeft `None` terug in plaats van een `KeyError` indien `sleutel` niet aanwezig
- `get(sleutel, standaardwaarde)`: geeft zelf een alternatief mee, `standaardwaarde`
- `in`: controleert of een sleutel bestaat

Code:

```
steden = {'Belgie': 'Brussel',
          'Nederland': 'Amsterdam'}
print(steden.get('Belgie'))
print(steden.get('Frankrijk'))
print(steden.get('Frankrijk', 'onbekend'))
print('Belgie' in steden)

# steden['Frankrijk'] geeft een KeyError
```

Output:

```
Brussel
None
onbekend
True
```


KU LEUVEN

Sorteren van itereerbare objecten

- Itereerbare objecten (sets, lijsten, woordenboeken, etc.) kunnen gesorteerd worden gegeven dat de elementen vergelijkbaar zijn:
  - Dit geldt oa. voor getallen (=,<,>,=<,>=) en strings (alfabetisch)
- 2 mogelijkheden:
  - `sorted()`: ingebouwde Python **functie** (zoals `len()` of `type()`) die een gesorteerd object teruggeeft
  - `sort()`: ingebouwde **methode** van lijsten - eens opgeroepen is het object zelf gesorteerd en krijgt het een nieuwe volgorde

KU LEUVEN

Voorbeeld sorteren

Code:

```
lijst = [0, 4, 1, -5, 2]
print(lijst)
print(sorted(lijst))
print(lijst)
lijst.sort()
print(lijst)
```

Output:

```
[0, 4, 1, -5, 2]
[-5, 0, 1, 2, 4]
[0, 4, 1, -5, 2]

[-5, 0, 1, 2, 4]
```

Originele lijst:

Gesorteerde lijst (niet permanent):

Lijst is hetzelfde gebleven:

Nu is de lijst wel definitief aangepast:

KU LEUVEN

Oefening 1.3: voorspel de output

```
s = {1, 2, 2, 3, 3, 3}
print(len(s), sorted(s))

l = [10, 20, 30, 40]
print(l[1], l[-1], l[1:3])

d = {'a': 1, 'b': 2}
print(d['a'], d.get('c'))
```

1. **Voorspel eerst de output (30 s)**
2. **Dan runnen en verbeteren in de notebook**

4. Operatoren en bewerkingen

1A.6-operatoren.ipynb

KU LEUVEN

Introductie

- Python bevat allerlei basisbewerkingen die kunnen uitgevoerd worden met operatoren
- Interpretatie anders per type van variabele/waarde waarop toegepast:
  - `a + b`
  - Optellen van strings resulteert in concatenatie
  - Optellen van getallen geeft de som
  - Optellen van lijsten voegt de elementen achter elkaar in 1 lijst (zoals `.extend()`)
- De operatoren moeten altijd met dezelfde types gebruikt worden:
  - Dus niet: string + integer (bv. niet `2 + 'tekst'` maar wel `'2' + 'tekst'`)
- Zoals altijd: volgorde is belangrijk

KU LEUVEN

Rekenkundige operators

+	:	optellen
-	:	aftrekken
*	:	vermenigvuldigen
/	:	delen
//	:	gehele deling (afgerond naar onder)
%	:	rest van deling
**	:	exponent

Code:

```
a = 3
b = -10
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(10 // 3)
print(10 % 3)
print(b ** a)
```

Output:

```
-7
13
-30
-0.3
3
1
-1000
```


KU LEUVEN

Vergelijkende operatoren

==: is gelijk aan

!=: is niet gelijk aan

<: is kleiner dan

>: is groter dan

<=: is kleiner of gelijk aan

>=: is groter of gelijk aan

Code:

```
a = 10
b = -10
c = 5
d = 10
print(a == d)
print(a == b)
print(a != b)
print(a < b)
print(a >= b)
```

Output:

```
True
False
True
False
True
```

Vergelijken (vergelijkbare/sorteerbare) objecten zoals bv. getallen/strings

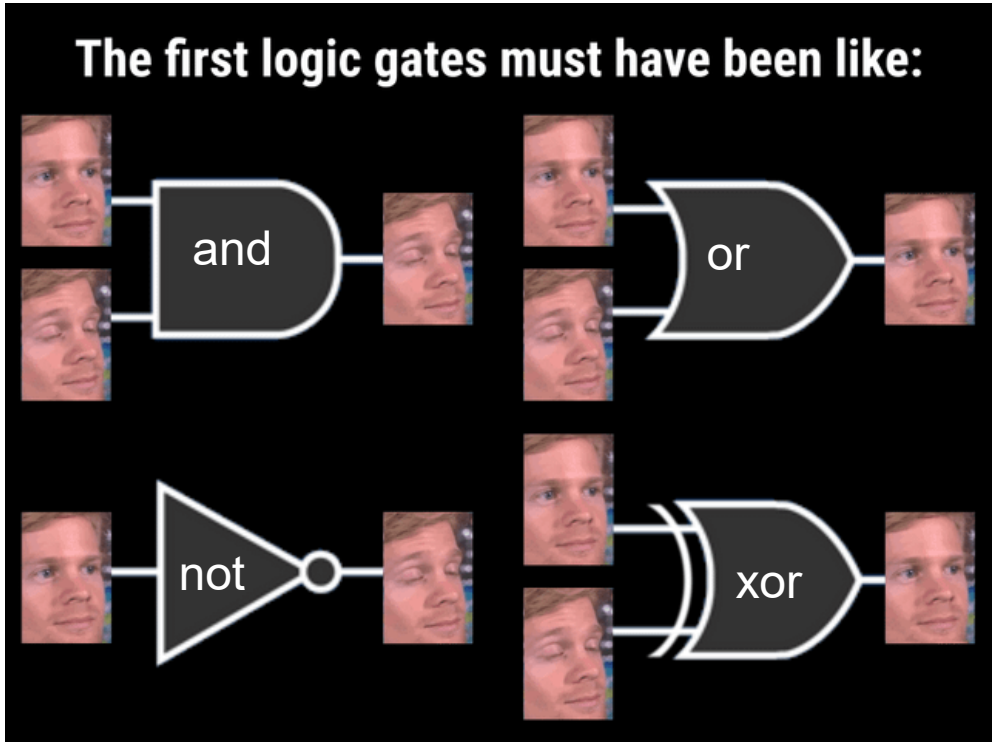
Logische operatoren

`not`:	het statement houdt niet
`or`:	minstens 1 van de statements geldt
`and`:	allebei de statements gelden
`is`:	object is gelijk aan ander object (plaats in het geheugen komt overeen)
`is not`:	object is niet gelijk aan ander object

- Gebruiken de Boolean uitkomst van een statement (bv. `a > 2`)
- Worden vooral gebruikt in `if`-statements (zie later)

KU LEUVEN

Logische operatoren



Code:

```
a = True
b = False
print('a and b', a and b)
print('a or b', a or b)
print('a is b', a is b)
print('a is not b', a is not b)
print('not a', (not a))
print('not b', (not b))
```

Output:

```
a and b False
a or b True
a is b False
a is not b True
not a False
not b True
```

https://www.reddit.com/r/LogicGateMemes/comments/d2nr07/the_first_logic_gates/

Gelijkheid

- `is` operator: controleert of 2 objecten gelijk zijn
- `==` operator: controleert of 2 waarden gelijk zijn

Code:

```
a = [1, 2, 3]
b = [1, 2, 3]
c = 10
d = 10
e = 'Johannes'
f = 'Johannes'

print(a == b)
print(a is b)
print(c == d)
print(c is d)
print(e == f)
print(e is f)
```

Output:

```
True
False
True
True
True
True
```

- De inhoud van de lijsten is hetzelfde
- De objecten zijn niet hetzelfde
- De integer waarden en objecten zijn wel hetzelfde:
  - Meestal houdt onze Python implementatie deze integers/strings bij als hetzelfde object op dezelfde geheugenplaats (dit is mogelijk bij simpele types zoals integers)

KU LEUVEN

Volgorde

Prioriteit	Operator
laagste	or
	and
	not
	==, !=, <, <=, >, >=, is, is not
	+, -
	*, /, //, %
hoogste	**

((4**3) * 2) // 3) + 5

4 – (((2 + 4)**2) % 5))

Code:

```
print(5 + 2 * 4 ** 3 // 3)
print(4 - (2 + 4)**2 % 5)
```

Output:

```
47
3
```


KU LEUVEN

Oefening 1.4: voorspel de output

```
print(7 / 2, 7 // 2, 7 % 2)
print(-7 // 2, -7 % 2)

a = [1, 2]
b = [1, 2]
print(a == b, a is b)

x = 5
print(0 < x < 10, not x > 3)
```

1. **Voorspel eerst (30 s)**
2. **Dan runnen en verbeteren**

5. Statements en control flow

KU LEUVEN

Soorten statements

- Toewijzigen:
  - Vullen of aanpassen van variabelen, bv. `a = 10`
- Oproepen van functies/methodes:
  - Zie later (maar denk aan bv. `str()`, `int()`, of `print()` die standaard in Python zitten)
- Conditionele uitvoering:
  - If/then/else
- Herhaling:
  - For-loops
  - While-loops

KU LEUVEN

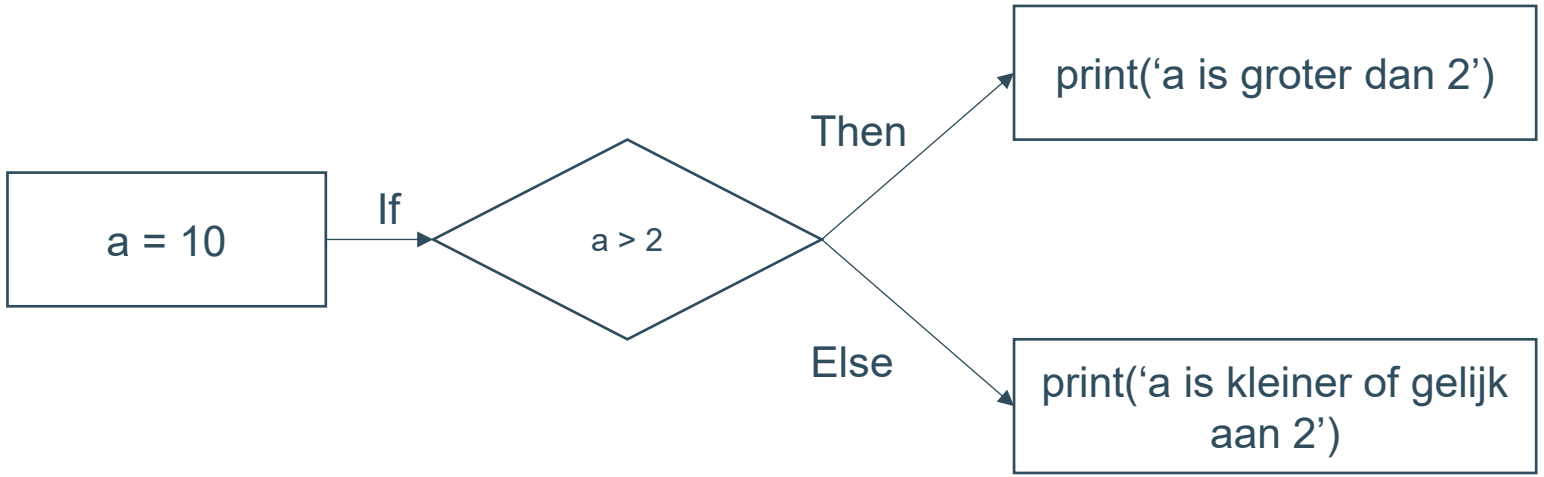
If-then-else

1A.7-conditionals.ipynb

KU LEUVEN

If-then-else

- `if`: conditionele uitvoering die een boolean uitkomst verwacht om voorwaarden te controleren
- `else`: is optioneel en vangt alle uitvoering op wanneer de conditie(s) niet voldaan is/zijn




```
a = 10

if a > 2:
    print('a is groter dan 2')
else:
    print('a is kleiner dan of gelijk aan 2')
```


KU LEUVEN

Indents

- Inspringen om control flow aan te duiden en ‘diepte’ van uitvoering
- Voor functies, for-loops, if/then, etc.
- Met een tab of 4 spaties (standaard 4 spaties in Spyder, geen verschil in Jupyter notebooks)

Code:

```
a = 10

if a > 2:
    print('\tBinnen het if-statement')
    print('\ta is groter dan 2')
print('Dit komt na het if-statement')
```

Output:

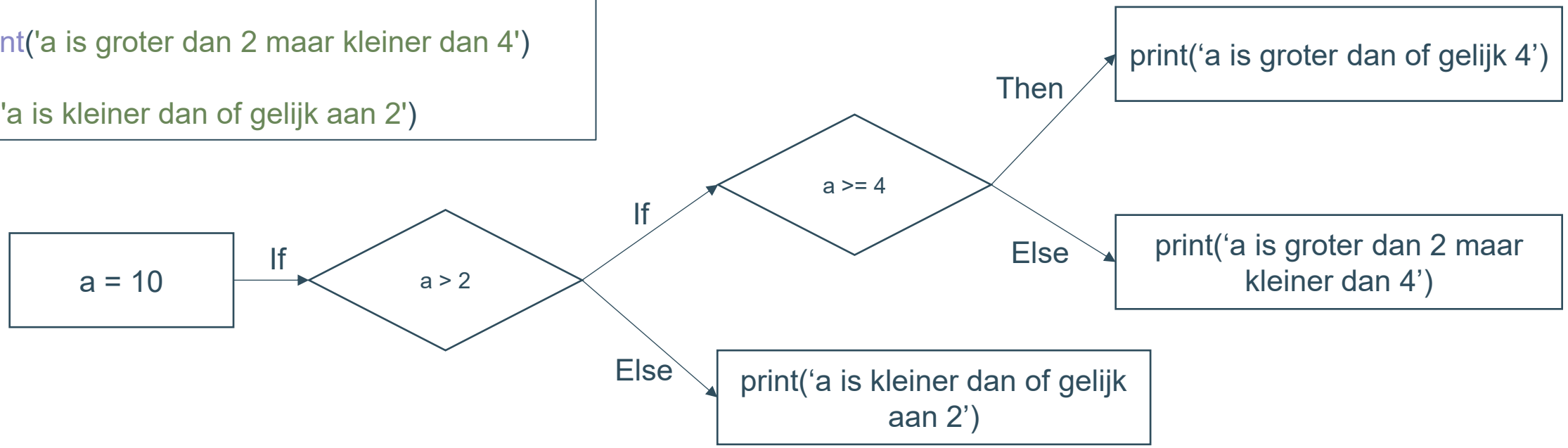
```
        Binnen het if-statement
        a is groter dan 2
Dit komt na het if-statement
```


KU LEUVEN

Geneste `if`

- If statements kunnen ook binnen andere if-statements voorkomen

```
if a > 2:
    if a >= 4:
        print('a is groter dan of gelijk aan 4')
    else:
        print('a is groter dan 2 maar kleiner dan 4')
else:
    print('a is kleiner dan of gelijk aan 2')
```

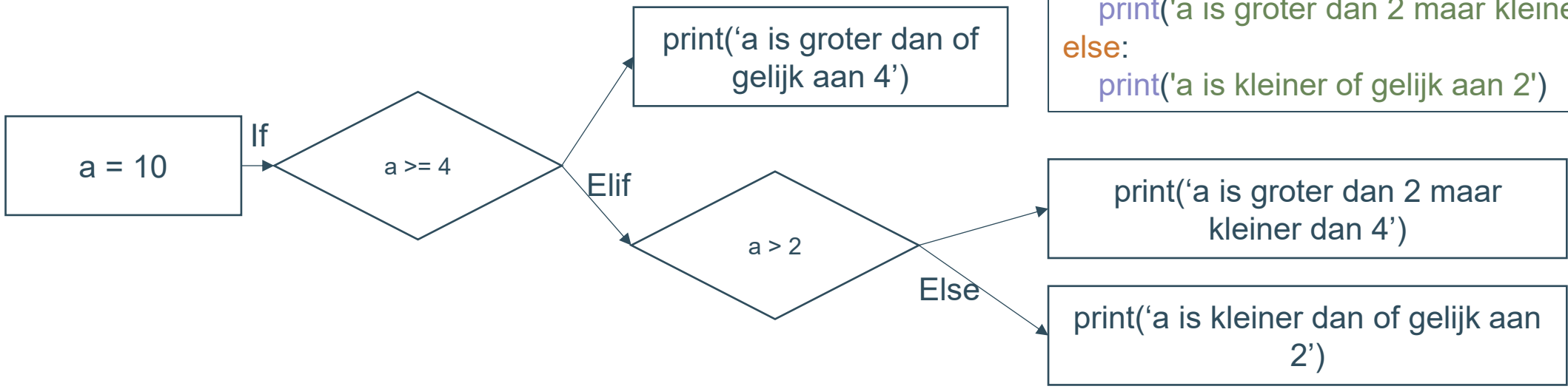


KU LEUVEN

If-elif-else

- Indien er meerdere condities zijn: `elif` (else if) kan ook gebruikt worden
- Ideaal voor intervallen
- Overzichtelijker

```
if a >= 4:
    print('a is groter dan of gelijk 4')
elif a > 2:
    print('a is groter dan 2 maar kleiner dan 4')
else:
    print('a is kleiner of gelijk aan 2')
```



KU LEUVEN

Geneste if/elif

- Opletten dat condities elkaar niet tegenspreken

```
if a >= 10:
    if a < 5:
        print('dit wordt nooit uitgevoerd')
    else:
        print('a is groter dan of gelijk aan 10')
else:
    print('a kleiner dan 10')
```

De geneste `if` wordt nooit uitgevoerd

```
if a >= 10:
    print('a is groter dan of gelijk aan 10')
elif a <= 5:
    print('a is kleiner dan of gelijk aan 5')
else:
    print('a is kleiner dan 10 maar groter dan 5')
```

Deze `elif` wordt wel uitgevoerd. Hier is de uitkomst duidelijker maar houden de intervallen misschien niet meteen steek (beter >=10, dan > 5, dan else)

KU LEUVEN

Logische operatoren

- Zorgen ervoor dat condities aan elkaar worden gelijmd
- Worden geëvalueerd van links naar rechts:
  - Zet conditie die hoogste kans heeft om te falen eerst bij AND
  - Zet conditie die hoogste kans heeft om waar te zijn eerst bij OR

Code:

```
a = 30
b = 10
c = False

if a > 20 and b < 2:
    print('Eerste deel')
elif not c or a == 10:
    print('c houdt niet')
else:
    print('Else')
```

Output:

```
c houdt niet
```

Code:

```
a = 10
b = 10
if a == b:
    print('a == b')
if a is b:
    print('a is b')
```

Output:

```
a == b
a is b
```


KU LEUVEN

Iteraties

1A.8-iteraties.ipynb

KU LEUVEN

While-loops

- Blijven uitgevoerd worden zolang de meegegeven conditie houdt
- Handig als er geen vast aantal mogelijkheden moet overlopen worden
  - Bv. een bepaald getal vinden in een lijst (weten niet wanneer we het tegenkomen)
- Heel foutengevoelig en kan makkelijk leiden tot oneindige lussen
- Kan afgesloten worden met een `else` die wordt uitgevoerd als de while (zonder `break`) is afgelopen

Code:

```
a = 3

while a > 0:
    print(a)
    a = a - 1
```

Output:

```
3
2
1
```


KU LEUVEN

While loops

- Overlopen van een lijst die stapsgewijs wordt leeggemaakt
- Hier wordt `else` gebruikt
- Let op: verwijder `.pop()` en het programma blijft eeuwig verderlopen

Een lijst/verzameling is ‘waar’ zolang die niet leeg is

Code:

```
b = [1, 'a', 3, 'd’]

while b:
    print(b)
    b.pop()
else:
    print('Lijst is nu leeg')
```

Output:

```
[1, 'a', 3, 'd']
[1, 'a', 3]
[1, 'a']
[1]
Lijst is nu leeg
```


KU LEUVEN

For-loops

- Overlopen een vast/vooraf gekend aantal mogelijkheden
- Overlopen itereerbare objecten zoals lijsten, sets, strings, ranges, files, etc.
- Voor de itereerbare objecten wordt een iterator-object aangemaakt dat hun inhoud element per element overloopt, maar dit blijft in de achtergrond

`in` wordt gebruikt in combinatie met een nieuwe variabele (hier `a`) en het te overlopen itereerbaar object, de lijst `[1,2,3]`

Code:

```
for a in [1, 2, 3]:
    print(a)
```

Output:

```
1
2
3
```


KU LEUVEN

For-loops voor dictionaries

Code:

```
b = {'a': 10, 'b': 20, 'c': 30}
for (sleutel, waarde) in b.items(): # geeft de key-value paren
    print(sleutel, '-', waarde)

for waarde in b.values(): # geeft de waardes
    print(waarde)

for sleutel in b: # gewoon een dict overlopen geeft de sleutels terug
    print(sleutel)
```

Output:

```
a - 10
b - 20
c – 30

10
20
30

a
b
c
```


KU LEUVEN

Range functie

- Laat toe om makkelijk integer reeksen te maken
- Maakt geen lijst aan, maar een itereerbaar range object
- Kan omgezet worden naar een lijst (met `list()`) voor andere doeleinden
- Heeft een begin, einde en interval/sprong argument

Merk op: bovengrens

wordt zoals altijd niet meegenomen

Code:

```
for i in range(0, 6):
    print(i)
print('--')
for i in range(6, 0, -2):
    print(i)
```

ondergrens bovengrens sprong

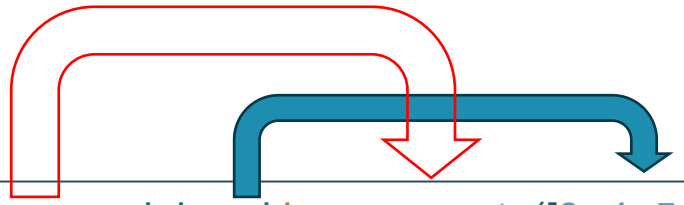
Output:

```
0
1
2
3
4
5
--
6
4
2
```


KU LEUVEN

enumerate

- Maakt automatisch een teller aan voor de iteraties over een itereerbaar object
- Resultaat is nu een tupel met 2 elementen: (volgnummer, element)



Code:

```
for volg_nummer, inhoud in enumerate([3, 4, 5, 6]):
    print(volg_nummer, inhoud, sep=' : ')
print('----')
for volg_nummer, inhoud in enumerate(set([2, 8, 9, 9])):
    print(volg_nummer, inhoud, sep=' : ')
```

Output:

```
0 : 3
1 : 4
2 : 5
3 : 6
----
0 : 8
1 : 9
2 : 2
```


KU LEUVEN

break en continue

- `break` laat het toe uit een iteratie te springen
- `continue` laat het toe om (stappen in) een iteratie over te slaan en de volgende iteratie in te zetten

Code:

```
for i in range(0, 10):
    if i == 2:
        break
    print('i:', i)

f = 2
while f < 7:
    f += 1
    if f == 3:
        continue
    print('f:', f)
```

Output:

```
i: 0
i: 1
f: 4
f: 5
f: 6
f: 7
```


KU LEUVEN

break en continue (2)

Code:

```
even_getallen_gevonden = 0
getal = 10

while even_getallen_gevonden < 5:
    if getal % 2 == 1:  # Controleer of het getal oneven is
        getal += 1
        continue  # Ga door met de volgende iteratie als het getal oneven is

    print(getal)
    even_getallen_gevonden += 1
    getal += 2  # Ga naar het volgende even getal
```

Output:

```
10
12
14
16
18
```

Speel zelf eens met de beginwaarde van `getal`

```
getallen = [5, 8, -3, 10, -2, 15]

for getal in getallen:
    if getal < 0:
        print(f"Eerste negatieve getal gevonden: {getal}")
        break  # Verlaat de lus zodra een negatief getal is gevonden
```

```
Eerste negatieve getal gevonden: -3
```


KU LEUVEN

Oefening 1.5: voorspel de output

```
score = 85
if score >= 60:
    print('geslaagd')
elif score >= 85:
    print('onderscheiding')

for i in range(1, 4):
    print(i)
```

1. **Voorspel eerst (30 s)**
2. **Dan runnen en verbeteren**

6. Comprehensions

1A.9-comprehensions.ipynb

KU LEUVEN

Introductie

- Samentrekkingen (eigen vertaling van comprehension) laten toe lijsten, sets en dictionaries aan te maken met 1 lijn code
- Kunnen zo alles wat een `map` en `filter` functie kunnen en meer (zie 1.9)
- Handig om bv. lijsten aan te passen, dictionaries te vullen met standaardwaarden voor de sleutels, etc.

KU LEUVEN

Lijstsamentrekkingen

- De gekende vierkante haken worden opgevuld met elementen van een for-loop met eventuele condities (zodat we filteren)

Code:

```
eerste_lijst = []
for i in range(5):
    eerste_lijst.append(i ** 2)
print('Eerste lijst:', eerste_lijst)

# korter met comprehension
een_lijn_lijst = [i ** 2 for i in range(5)]
print('1-lijn lijst:', een_lijn_lijst)


# met condities (zoals een filter)
een_andere_lijst = [i ** 2 for i in range(10) if i % 2 == 0]
print('Kwadraten van even getallen onder tien:', een_andere_lijst)
```

`if` staat hier achteraan en werkt als filter

Output:

```
Eerste lijst: [0, 1, 4, 9, 16]

1-lijn lijst: [0, 1, 4, 9, 16]

Kwadraten van even getallen onder tien: [0, 4, 16, 36, 64]
```


KU LEUVEN

Lijstsamentrekkingen

- Werkt ook met functies zodat we kunnen mappen en condities vooraan zodat we alternatieve uitkomsten krijgen

`if` staat hier vanvoor om verschillende uitkomst weg te schrijven

Code:

```
# met alternatieve waarde
een_laatste_lijst = [i ** 2 if (i % 2 == 0) else 0 for i in range(10)]
print('Een laatste lijst:', een_laatste_lijst)


# werkt met functies en strings
def doe_iets_met_een_element(element):
    return '__' + element.lower() + '__'

string = 'legendary'
een_nieuwe_string = [doe_iets_met_een_element(i) for i in string]
print('Een nieuwe string:', een_nieuwe_string)
```

Output:

```
Een laatste lijst: [0, 0, 4, 0, 16, 0, 36, 0, 64, 0]

Een nieuwe string: ['__l__', '__e__', '__g__', '__e__',
'__n__', '__d__', '__a__', '__r__', '__y__']
```


KU LEUVEN

Sets en dictionaries

- Werkt even zeer bij sets en dictionaries

Code:

```
# duplicaten vallen meteen weg in sets
woord_met_rare_tekens = 'joh@ann3s'
letters_uit_alfabet = {i for i in woord_met_rare_tekens if i.isalpha()}
print('Letters van het woord:', letters_uit_alfabet)


# net hetzelfde voor dictionaries maar met een dubbelpunt ertussen
woord_met_rare_tekens = 'joh@ann3s'
letter_count = {letter: woord_met_rare_tekens.count(letter) for letter in
woord_met_rare_tekens}
print('Aantal voorkomens van de letters:', letter_count)
```

Output:

```
Letters van het woord: {'j', 'o', 'h', 'a', 'n', 's'}


Letter: {'j': 1, 'o': 1, 'h': 1, '@': 1, 'a': 1, 'n': 2, '3': 1, 's': 1}
```


KU LEUVEN

Oefening 1.6: voorspel de output

```
print([x * x for x in range(5)])
print([x for x in range(10) if x % 3 == 0])

woorden = ['kat', 'hond', 'kat']
print({w for w in woorden})
```

1. **Voorspel eerst (30 s)**
2. **Dan runnen en verbeteren**

7. Strings

1A.10-strings.ipynb

KU LEUVEN

Introductie

- 1 van de basistypes van Python
- Worden overal gebruikt, ook om te debuggen (code na te kijken)
- Belangrijk de volle flexibiliteit te gebruiken

Code:

```
a = 'Johannes'
b = 'De Smedt'
print(a + b + b) # concatenatie
print(a + b + '_' + str(5)) # conversie van een integer
print(a[2:7:2]) # indexering
print(a[::-1]) # omgekeerd
print(len(a)) # lengte van de string
print('nnes' in a)
print('nnes' in b)
```

Output:

```
JohannesDe SmedtDe Smedt
JohannesDe Smedt_5
hne
sennahoJ
8
True
False
```


KU LEUVEN

Ingebouwde methodes

- Strings hebben een hele reeks aan methodes:
  - `capitalize()`: maakt 1ste letter van de string een hoofdletter, de rest kleine letters
  - `lower()`: maakt de hele string tot kleine letters
  - `upper()`: maakt de hele string tot hoofdletters
  - `title()`: maakt de 1ste letter van elk woord een hoofdletter
  - `count(`*`substring`*`)`: telt hoeveel keer de meegegeven *`substring`* voorkomt
  - `strip()`/`lstrip()`/`rstrip()`: verwijdert alle whitespaces (spaties, \t, \n) aan beide kanten/enkel links/enkel rechts

Strings zijn onveranderbaar

- Een string kan niet gewijzigd worden na het aanmaken (immutable)
- Methodes wijzigen de string niet, ze geven een nieuwe string terug
- Wil je het resultaat bewaren? Sla het opnieuw op in een variabele:

Code:

```
naam = ‘economie'
print(naam.capitalize())
print(naam) # variabele blijft ongewijzigd

naam = naam.capitalize()
print(naam)

# naam[0] = 'J' geeft een TypeError
```

Output:

```
Economie
economie


Economie
```


KU LEUVEN

Voorbeelden ingebouwde methodes

- Merk op: de `print()` functie laat ook toe strings met een komma aan elkaar te zetten met een spatie ertussen

Code:

```
a = 'programmeren Is Leuk'
b = '  programmeren is leuk als je het veel probeert     '
print(a.capitalize())
print(a.lower())
print(a.upper())
print(a.title())
print(b.count('ee'))
print('--', b.strip(), '--')
print('--', b.lstrip(), '--')
print('--', b.rstrip(), '--')
```

Output:

```
Programmeren is leuk
programmeren is leuk
PROGRAMMEREN IS LEUK
Programmeren Is Leuk
2
-- programmeren is leuk als je het veel probeert --
-- programmeren is leuk als je het veel probeert      --
--   programmeren is leuk als je het veel probeert --
```


KU LEUVEN

Andere ingebouwde methodes

- `find(substring)`: geeft de index van het eerste voorkomen van een *substring* (of -1 indien niet gevonden)
- `replace(te_vervangen, nieuwe_waarde)`: vervangt een substring (`te_vervangen`) met een nieuw stuk tekst (`nieuwe_waarde`)
- `rfind(substring)`: geeft de index van het laatste voorkomen van een *substring* (of -1 indien niet gevonden)
- `split(splitter)`: splitst een string op basis van een `splitter` string (optioneel) of whitespaces
- `rsplit(splitter)`: zoals `split()` maar begint aan de rechterkant
- `isalnum()`: controleert of alle tekens in de string alfanumeriek zijn
- `isalpha()`: controleert of alle tekens in de string letters zijn

KU LEUVEN

Voorbeelden ingebouwde methodes

Code:

```
a = '123 abc defgggg'
print(a.find('gg'))
print(a.replace('ggg', 'ffff'))
print(a.rfind('gg'))
print(a.split('abc'))
print(a.rsplit())
b = '123abc'
c = 'abc'
d = '@bc12'
print(b.isalnum())
print(b.isalpha())
print(c.isalnum())
print(c.isalpha())
print(d.isalnum())
print(d.isalpha())
```

Output:

```
11
123 abc deffffffg
13
['123 ', ' defgggg']
['123', 'abc', 'defgggg']

True
False
True
True
False
False
```

Geeft een integer terug
Geeft een string terug
Geeft een integer terug
Geven een lijst terug
Geven een boolean terug

Samenvoegen en controleren

- `join()`: plakt een lijst van strings samen met een scheidingsteken
- `split()` en `join()` zijn elkaars tegengestelde
- `startswith()` en `endswith()`: controleren begin of einde van een string

Code:

```
woorden = ['Python', 'is', 'leuk']
print(' '.join(woorden))
print('-'.join(woorden))
zin = ' '.join(woorden)
print(zin.split(' '))
print(zin.startswith('Python'))
print(zin.endswith('?'))
```

Output:

```
Python is leuk
Python-is-leuk

['Python', 'is', 'leuk']
True
False
```


KU LEUVEN

Strings printen

- Kan met de ingebouwde `print` functie
- Aaneenkoppelen van strings kan met een komma die ook een enkele spatie tussenvoegt (of een andere separator die je zelf kiest)
- Variabelen kunnen ook ingevoegd worden mbv een voorzetsel f’ en {} in de string
  - Dit kan ook buiten de `print`-functie
- Verder kunnen strings placeholders hebben voor variabelen/waardes en voorgemaakte formattering (bv. het aantal cijfers na de komma)

KU LEUVEN

Voorbeelden strings printen

'sep' is een argument van de print functie, het geeft hier mee welke separator moet gebruikt worden (moet op laatste plaats komen en expliciet aangeduid met `sep=`)

Code:

```
a = 2
b = 'Johannes'
print('Zoals voorheen gezien', a, b)
print('Met een andere', 'separator ertussen', sep='---___---')
print('Concatenatie vereist juiste types ' + str(a))
print(f'{b} geeft {a} vakken')
print('Ik kocht %d bananen in de winkel aan %.1f euro' % (2, 0.35))
```

Output:

```
Zoals voorheen gezien 2 Johannes
Met een andere---___---separator ertussen
Concatenatie vereist juiste types 2
Johannes geeft 2 vakken
Ik kocht 2 bananen in de winkel aan 0.3 euro
```

De `f'` als voorzetsel van een string duidt aan dat er variabelen gebruikt worden in de string

De waarden komen er achteraan bij als een tupel (merk op hoe 0.35 wordt afgerond tot 1 cijfer na de komma)

`%` wordt gebruikt om waardes in de string te pluggen:

- `f` voor floating point waarde die ook een precisie kan krijgen (hier 1 getal na de komma)
- `s` voor string
- `d` voor integer

KU LEUVEN

De `format()`-methode

- Is beter gestructureerd dan het gebruik van %
- Wordt toegepast op een string met vervangingsvelden tussen {}
- Argumenten zijn genummerd en kunnen zo op de juiste plaats gezet worden (of herhaald)

Zelfs lijsten kunnen gebruikt worden

Code:

```
a = 100
b = 0.222
c = 'aandelen'
print('Ik kocht {0} {1} aan {2}'.format(a, c, b))
print('Ik kocht {2} {1} aan {0}'.format(a, c, b))
print('Ik kocht {0[0]} {0[1]} aan {1}'.format([100, 'aandelen'], 0.12))
print('Ik kocht een aandeel aan {0:.2f}'.format(10200.2211))
```

Output:

```
Ik kocht 100 aandelen aan 0.222
Ik kocht 0.222 aandelen aan 100
Ik kocht 100 aandelen aan 0.12
Ik kocht een aandeel aan 10200.22
```


KU LEUVEN

Aantal spaties

- Door een getal mee te geven voor de komma (of gewoon een getal) in de `format`-functie kunnen we een vast aantal tekenplaatsen/whitespaces innemen om getallen af te drukken
- Handig om uit te lijnen

```
print('Ik kocht een aandeel aan {0:20.2f}'.format(10200.2211))
for i in range(0, 11):
    print('i is {0:2}'.format(i))
```

We gebruiken 20 plaatsen om dit getal te printen

Alle getallen gebruiken 2 plaatsen, zodat ze mooi onder elkaar komen

```
Ik kocht een aandeel aan          10200.22
i is  0
i is  1
i is  2
i is  3
i is  4
i is  5
i is  6
i is  7
i is  8
i is  9
i is 10
```


KU LEUVEN

f-strings met formattering

- Zelfde formattering als bij `.format()`: breedte (`10`), precisie (`.2f`)
- Uitlijning binnen die breedte: `<` links, `>` rechts, `^` gecentreerd
- Optioneel opvulteken voor het uitlijnteken, bv. `*^10`

Code:

```
naam = 'appel'
prijs = 12.3456
print(f'|{naam:<10}|')   # links
print(f'|{naam:>10}|')   # rechts
print(f'|{naam:^10}|')   # midden
print(f'|{naam:*^10}|')  # opvulteken
print(f'|{prijs:>10.2f}|')
```

Output:

```
|appel     |
|     appel|
|  appel   |
|**appel***|
|     12.35|
```


KU LEUVEN

Input via de console

- De `input()`-functie laat toe om strings in te lezen
- Makkelijk om een interactief programma te maken
- De ingelezen string zal wel moeten herwerkt worden om binnen het programma bruikbaar te zijn (bv. omzetten naar integers)

Code:

```
ingelezen_string = input()
print('---', ingelezen_string, '---')
```

Input: ik schrijf een zin in de console

Output: --- ik schrijf een zin in de console ---

```
ingelezen_string = input('Geef een nummer')
print('Omgezet:', int(ingelezen_string))
```

Input: 55

Output: Omgezet: 55

KU LEUVEN

Groter voorbeeld

Code:

```
zin = "Vanavond eten we spaghetti"
print(zin)
letters = set(zin)
teller = 0
output_string = ''
for teken in letters:
    output_string += "{0:3s} komt {1:3d} keer voor \t".format(teken, zin.count(teken))
    teller += 1
    if teller % 3 == 0:
        print(output_string)
        output_string = ''
```

Output:

```
Vanavond eten we spaghetti
v   komt   1 keer voor      i   komt   1 keer voor      g   komt   1 keer voor
o   komt   1 keer voor          komt   3 keer voor      s   komt   1 keer voor
t   komt   3 keer voor      e   komt   4 keer voor      p   komt   1 keer voor
a   komt   3 keer voor      h   komt   1 keer voor      V   komt   1 keer voor
w   komt   1 keer voor      n   komt   3 keer voor      d   komt   1 keer voor
```


KU LEUVEN

Oefening 1.7: voorspel de output

```
s = "python"
s.upper()
print(s)

print("a,b,c".split(","))
print("-".join(["a", "b"]))

pi = 3.14159
print(f"{pi:.2f}")
```

1. **Voorspel eerst (30 s)**
2. **Dan runnen en verbeteren**

8. Functies

1A.11-functies.ipynb

KU LEUVEN

Introductie

- Functies kunnen zelf gedefinieerd en gebruikt worden:
  - Om code overzichtelijk te houden
  - Om te herbruiken op verschillende plaatsen
  - Om logica af te schermen
- Functies hebben inputs (argumenten) en mogelijk outputs
- Functie gekoppeld aan bepaald object: methode
- Veel ingebouwde functies zijn al aan bod gekomen:
  - `print()`
  - `str()/int()/bool()/float()`
  - `len()`

Voorbeeldfunctie

1. `def` duidt een zelfgemaakte functie aan
2. Wordt gevolgd door de naam van de functie
3. De argumenten volgen tussen haakjes
4. Als de functie iets teruggeeft gebeurt dit via `return`

Code:

```
def geef_lengte_van_strings(string_1, string_2):
    return len(string_1) + len(string_2)

a = 'Johannes'
b = 'De Smedt'
totale_lengte = geef_lengte_van_strings(a, b)
print(f'Lengte van {a} en {b} opgeteld:', totale_lengte)
```

1 2 3 4

De functie moet uiteraard gedefinieerd zijn voor gebruik (dus boven dit statement – in JN: in een cell die al is uitgevoerd)

Output:

```
Lengte van Johannes en De Smedt opgeteld: 16
```


KU LEUVEN

Variabelen binnen functies

- Variabelen hebben een ‘scope’ waarbinnen ze gelden:
  - Variabelen aangemaakt binnen een functie bestaan niet daarbuiten:
    - Tenzij het keyword `global` wordt gebruikt bij het aanpassen van de variabelen binnen de functie
  - Variabelen van het hoofdprogramma bestaan overal:
    - Vermijd om deze variabelen binnen functies te gebruiken tenzij ze worden meegegeven als argumenten (en worden dan best gekopieerd indien geen basis type):
      - Kunnen overschreven worden binnen functie zonder dat implicaties daarbuiten gekend zijn (voor bv. een andere functie die de variabele gebruikt)
    - Kunnen eventueel expliciet aangeduid worden met het keyword `global`
      - Zie later recursie

Variabelen binnen functies

- Om fouten te vermijden moet de structuur van het programma doorgrond worden:

Code:

Functieblok

```
def geef_lengte_van_strings(string_1, string_2):
    print('a is hier ook beschikbaar:', a)
    variabele_binnen_functie = 'functiestring'
    print(variabele_binnen_functie)
    return len(string_1) + len(string_2)
```

```
a = 'Johannes'
b = 'De Smedt'
totale_lengte = geef_lengte_van_strings(a, b)
print(f'Lengte van {a} en {b} opgeteld:', totale_lengte)
print(variabele_binnen_functie)
```

Hoofdblok

Output:

```
a is hier ook beschikbaar: Johannes
functiestring
Lengte van Johannes en De Smedt opgeteld: 16

Traceback (most recent call last):
  File “~/SOEP1/1-functies.py", line 26, in <module>
    print(variabele_binnen_functie)
NameError: name 'variabele_binnen_functie' is not defined
```


KU LEUVEN

Voorbeeld van slecht gebruik `global`

```
# Slecht voorbeeld: Gebruik van een globale variabele die de code
complex maakt (en onbetrouwbaar)

# Globale variabele
teller = 0

# Functie die de globale variabele verhoogt
def verhoog_teller():
    global teller
    teller += 1

# Functie die de globale variabele gebruikt om iets te doen
def doe_iets():
    if teller % 2 == 0:
        print("Teller is even")
    else:
        print("Teller is oneven")

# Hoofdprogramma
verhoog_teller()
doe_iets()
```

In dit voorbeeld is `doe_iets()` afhankelijk van de andere functie

```
# Goed voorbeeld: Gebruik van argumenten om de
afhankelijkheden duidelijker te maken

# Functie die de teller verhoogt en de nieuwe waarde retourneert
def verhoog_teller(teller):
    return teller + 1

# Functie die controleert of een getal even of oneven is
def is_even(getal):
    return getal % 2 == 0

# Hoofdprogramma
teller = 0
teller = verhoog_teller(teller)
if is_even(teller):
    print("Teller is even")
else:
    print("Teller is oneven")
```

In dit voorbeeld gebruiken beide functies enkel de meegegeven argumenten

KU LEUVEN

Argumenten

- Moeten altijd ingevuld worden
- Er kunnen niet meer argumenten meegegeven worden dan aangegeven in de functiedefinitie
- Bij oproep van een functie kan de naam van het argument meegeven worden waardoor de volgorde van de argumenten (deels) vervalt:
  - De namen worden ook getoond in code editors zoals PyCharm of VSC
- Default waarden zijn mogelijk (en maken het argument wel optioneel):
  - Moeten altijd na de argumenten zonder default waarde komen

KU LEUVEN

Voorbeeld argumenten

Argument string_3 heeft een default waarde ‘johannes’

Volgorde volgt niet die van functiedefinitie omdat de argumentennamen worden meegegeven

Code:

```
def print_strings(string_1, string_2, string_3='johannes'):
    return string_1 + string_2 + string_3

print('Dit mag:', print_strings(string_2 = 'Johannes', string_1 = 'De Smedt'))
```

Output:

```
Dit mag: De SmedtJohannesjohannes
```

Code:

```
def print_strings(string_1='johannes', string_2, string_3):
    return string_1 + string_2 + string_3
```

Mag niet: argumenten zonder default waarden eerst

KU LEUVEN

Opgelet

Links zijn er bijwerkingen voor het hoofdprogramma
Rechts wordt alles gescheiden

Code:

```
def verwijder_eerste_element_uit_lijst(lijst_in):
    del lijst_in[0]

een_lijst = [0, 1, 2, 3]
print('Lijst in het begin:', een_lijst)
verwijder_eerste_element_uit_lijst(een_lijst)
print('Lijst na de functie:', een_lijst)
```

Output:

```
Lijst in het begin: [0, 1, 2, 3]
Lijst na de functie: [1, 2, 3]
```

Code:

```
def geef_lijst_zonder_eerste_element_uit_lijst(lijst_in):
    lijst_2 = lijst_in.copy()
    del lijst_2[0]
    return lijst_2

een_lijst = [0, 1, 2, 3]
print('Lijst in het begin:', een_lijst)
nieuwe_lijst = geef_lijst_zonder_eerste_element_uit_lijst(een_lijst)
print('Lijst na de functie:', een_lijst)
print('Resultaat van de functie:', nieuwe_lijst)
```

Output:

```
Lijst in het begin: [0, 1, 2, 3]
Lijst na de functie: [0, 1, 2, 3]
Resultaat van de functie: [1, 2, 3]
```


KU LEUVEN

Opgelet (2)

- Voor eenvoudige types wordt wel de waarde en niet de verwijzing meegegeven zoals bij de lijst hiervoor:

Code:

```
def verdubbel_integer(x):
    print('x voor:', x)
    x = x * 2
    print('x na:', x)

y = 10
print('y eerst:', y)
verdubbel_integer(y)
print('y na functie:', y)
```

Output:

```
y eerst: 10
x voor: 10
x na: 20
y na functie: 10
```


KU LEUVEN

return

```
def vind_lager_getal_in_lijst_1(lijst_in, getal) -> int:
    for element in lijst_in:
        if element < getal:
            return element

def vind_lager_getal_in_lijst_2(lijst_in, getal) -> None:
    for element in lijst_in:
        if element < getal:
            print('Gevonden:', getal)
            return

lijst_1 = [0, 5, 100, 2]
getal = 10
output = vind_lager_getal_in_lijst_1(lijst_1, getal)
vind_lager_getal_in_lijst_2(lijst_1, getal)
```

We gebruiken type hinting om aan te geven welk outputtype we verwachten:

- `int` voor de eerste functie
- `None`: geen resultaat verwacht voor de tweede

Hier stopt functie door een resultaat te geven (de rest van de `for` wordt dan niet verder uitgevoerd)

Hier stopt functie door een leeg resultaat te geven (ook hier wordt de `for` mogelijks niet volledig uitgevoerd)

`return` wordt in beide gevallen gebruikt om voortijdig een for-loop af te sluiten

KU LEUVEN

Meerdere waarden teruggeven in functie

- Meerdere waarden teruggeven: scheid ze met een komma
- Python bundelt ze in een tuple
- Je kan die tuple meteen uitpakken in aparte variabelen

Code:

```
def deel_met_rest(teller: int, noemer: int):
    return teller // noemer, teller % noemer

resultaat = deel_met_rest(17, 5)
print(resultaat)

geheel, rest = deel_met_rest(17, 5)
print(geheel)
print(rest)
```

Output:

```
(3, 2)
3
2
```


KU LEUVEN

Ongespecifieerd aantal parameters

- Gebruik van een asterisk voor een parameter maakt dat deze wordt geïnterpreteerd als een tupel
- Zo kan een ongespecificeerd aantal parameters meegegeven worden
  - Dit is bv. hoe de `print()`-functie meerdere strings achter elkaar plakt

Code:

args is een tuple

Andere parameters kunnen ook nog meegegeven worden, maar enkel met hun expliciete vermelding (i.e. argumentnaam)

```
def som_getallen(*args, minimum):
    print('Aantal getallen meegegeven:', len(args))
    return sum(args) + minimum

print('Uitkomst 1:', som_getallen(1, 2, 3, 4, minimum=10))
print('Uitkomst 2:', som_getallen(1, 4, minimum=100))
```

Output:

```
Aantal getallen meegegeven: 4
Uitkomst 1: 20
Aantal getallen meegegeven: 2
Uitkomst 2: 105
```


KU LEUVEN

Documenteren

- Functies kunnen overal worden ingebouwd, bv. ook in externe modules die we later gebruiken
- Daarom is het belangrijk dat er informatie voorzien is over wat de functie precies doet en welke argumenten verwacht worden
- Type hinting:
  - Python heeft geen expliciete typering
  - We kunnen wel aanduiden welke types we verwachten als argumenten
  - We kunnen ook aanduiden welk type de functie normaal zal teruggeven
  - Louter informatief

Voorbeeld documentatie



De string tussen 3 ” wordt in code editors meegegeven als we over de functie hoveren met de muis

Code:

```
def som_van_getallen(getal_1: 'integer', getal_2: 'integer') -> 'integer':
    """Deze functie telt 2 getallen op"""
    return getal_1 + getal_2


print(som_van_getallen(1, 4))
print(som_van_getallen('een', ' string!'))
print(som_van_getallen.__annotations__)
```

Type hinting van argumenten:
`: 'integer'`
Type hinting van output:
`-> 'integer'`

De rest van de info kan opgevraagd worden via het .__annotations__ attribuut van de functie

Output:

```
5
een string!
{'getal_1': 'integer', 'getal_2': 'integer', 'return': 'integer'}
```

Let op, deze info is louter informatief. Men kan nog altijd bv. 2 strings meegeven hoewel ‘integer’ vermeld wordt voor beide argumenten

KU LEUVEN

Oefening 1.8: voorspel de output

```
def dubbel(x):
    print(x * 2)

y = dubbel(3)
print(y)

teller = 0
def verhoog():
    teller = 10
verhoog()
print(teller)
```

1. **Voorspel eerst (30 s)**
2. **Dan runnen en verbeteren**

9. Filteren en aanpassen met functies

1A.12-filter_map_functies.ipynb

KU LEUVEN

Introductie

- De `filter()` functie laat toe om een functie mee te geven als argument die een itereerbaar object kan filteren:
  - Argumenten: de filterfunctie en het itereerbaar object
  - Filtervoorwaarden zijn ofwel een lambda-functie of een eigen functie
- Het resultaat is een itereerbaar object dat terug kan omgezet worden:
  - Bv. het resultaat kan omgezet worden via `list()`
- In de achtergrond in C uitgevoerd, dus sneller dan standaard Python

Voorbeeld: filter positieve nummers weg

Code:

```
lijst = [0, 2, -5, 6, -3]

nieuwe_lijst = []
for i in lijst:
    if i < 0:
        nieuwe_lijst.append(i)
print('Nieuwe lijst:', nieuwe_lijst)

#  of korter met lambda functie
nieuwe_lijst = filter(lambda i: i < 0, lijst)
print('Nieuwe lijst:', list(nieuwe_lijst))
```

Output:

```
Nieuwe lijst: [-5, -3]

Nieuwe lijst: [-5, -3]
```

Definieer aan welke voorwaarden elk element (hier genaamd `i`, maar is een variabele als een andere) uit het itereerbaar object `lijst` moet voldoen om behouden te blijven

KU LEUVEN

Filter functies

- Eigen functie om te filteren:

We geven de functie mee als object
Maw, we roepen die niet zelf op (we gebruiken geen haakjes), de `filter`-functie doet dit zelf, telkens op elk element van het meegegeven itereerbaar object

Code:

```
def verwijder_a_voor_filter(letter: str) -> bool:
    return letter != 'a'

filteren_2 = filter(verwijder_a_voor_filter, 'programmeren')
print(filteren_2)
print(list(filteren_2))
```

Output:

```
<filter object at 0x7f9a18b44160>
['p', 'r', 'o', 'g', 'r', 'm', 'm', 'e', 'r', 'e', 'n']
```

Deze functie is kort en overzichtelijk en neemt als argument de elementen van het itereerbaar object
De output is een boolean die zegt of het element zal behouden blijven (een beetje tegen intuïtief, want filter behoudt wat als `True` terugkomt uit de meegegeven functie)

KU LEUVEN

Map functies

- Gelijkaardig aan `filter()` maar is nu bedoeld om een transformatie toe te passen op alle elementen van een itereerbaar object
- Geeft ook een itereerbaar object terug met daarin alle getransformeerde elementen
- Makkelijk om snel lijsten aan te passen
- Lambda-functies ook mogelijk

Voorbeeld: vierkantswortel

Code:

```
def neem_vierkantswortel(getal):
    return getal ** (1/2)

getallen = [1, 4, 16, 25]
uitkomst = map(neem_vierkantswortel, getallen)
print(list(uitkomst))

met_lambda = map(lambda x: x ** (1/2), getallen)
print(list(met_lambda))
```

Output:

```
[1.0, 2.0, 4.0, 5.0]

[1.0, 2.0, 4.0, 5.0]
```

Voorbeeld: meerdere resultaten

Dit is puur ter informatie en geen leerstof voor het examen

Lambda functies kunnen meerdere argumenten hebben

Code:

```
getallen = [1, 2, 3, 4]
getallen_2 = [2, 4, 5]
uitkomst = map(lambda x, y: x ** y, getallen, getallen_2)
print(list(uitkomst))

def dubbele_uitkomst(nummer):
    return nummer + 2, nummer + 3

print(list(map(dubbele_uitkomst, getallen)))
```

Output:

```
[1, 16, 243]

[(3, 4), (4, 5), (5, 6), (6, 7)]
```

Hier wordt een itereerbaar object teruggegeven dat als elementen paren heeft (eigenlijk tuples)

KU LEUVEN

Oefening 1.9: voorspel de output

```
r = filter(lambda x: x % 2 == 0, [1, 2, 3, 4])
print(r)
print(list(r))
print(list(r))

print(list(map(str.upper, ['an', 'bo'])))
```

1. **Voorspel eerst (30 s)**
2. **Dan runnen en verbeteren**

10. Objecten en klasses

1A.13-objecten.ipynb

KU LEUVEN

Opzet

- Klasses vormen een blauwdruk voor concepten in je programma:
  - Voorbeeld: blauwdruk van een student
    - Heeft **eigenschappen/attributen**: klasse-specifieke variabelen
      - Bv. een naam, een studentennummer, vakkenpakket
    - Heeft **methodes**: klasse-specifieke functies
      - Bv. aanpassen van het vakkenpakket, goedkeuren ISP, delibereren
- Een object is een instantiatie (dus een verwezenlijking) van een klasse:
  - Een specifiek studentobject:
    - Heeft ingevulde attributen specifiek aan de instantiatie maar conform de blauwdruk:
      - Naam = ‘Wilfried’, studentennummer = ‘10’ en vakkenpakket= een lijst van 3 vakken
    - Methodes van de klasse die resultaten geven voor het specifieke object:
      - geef_punt_voor_vak(), geef_gemiddelde_punt(), delibereer()

Voorbeeldklasse

We maken een **klasse** met naam `Student` aan

```
class Student:

    def __init__(self, naam_in: str, nummer_in: int, richting_in: str):
        self.naam = naam_in
        self.nummer = nummer_in
        self.richting = richting_in
        self.resultaten = {}

    def geef_naam(self) -> str:
        return self.naam

    def geef_punt_voor_vak(self, vak: str, punt: int):
        self.resultaten[vak] = punt

    def geef_gemiddelde_punt(self) -> float:
        punten = self.resultaten.values() # we krijgen alle punten terug als een itereerbaar object
        print(list(punten)) # dit moeten we nog in een lijst omzetten om de sum()-functie erover te kunnen gebruiken
        if self.resultaten.values(): # de dictionary is niet leeg
            return sum(list(punten)) / len(punten)
        else:
            return 0
```

Methodes van de klasse

Dit is de **constructor** – dit is een specifieke functie die wordt aangeroepen als we een instantie willen aanmaken met de oproep `Student()` (zie volgende slide).
De argumenten van `__init__()` en `Student()` komen overeen

De **attributen** van de klasse – deze worden best altijd aangemaakt in de constructor
Voorbeeld: `self.naam = naam_in` stopt het inputargument `naam_in` in de variabele `self.naam`

`self` is een variabele die de instantie van een klasse vertegenwoordigt. Deze wordt gebruikt om de variabelen later te kunnen ophalen en is altijd het eerste argument van elke methode van een klasse. Deze moet echter niet meegegeven worden bij het oproepen van de methode (zie volgende slide)

KU LEUVEN

Klasses gebruiken

student_1
naam = ‘Wilfried’ nummer = 10 richting = ‘HIRB’ resultaten = {}

student_2
naam = ‘Monique’ nummer = 11 richting = ‘HIRB’ resultaten = {}

- We maken instanties (objecten) aan volgens de blauwdruk:

```
student_1 = Student('Wilfried', 10, 'HIRB’)
student_2 = Student('Monique', 11, 'HIRB’)
```

→ We maken 2 objecten aan volgens de blauwdruk van de klasse `Student` met de attributen die in de constructor (`__init__()`-methode) aanwezig zijn (merk op dat `self` dus niet wordt meegegeven)

```
student_1.naam
student_1.geef_naam()
```

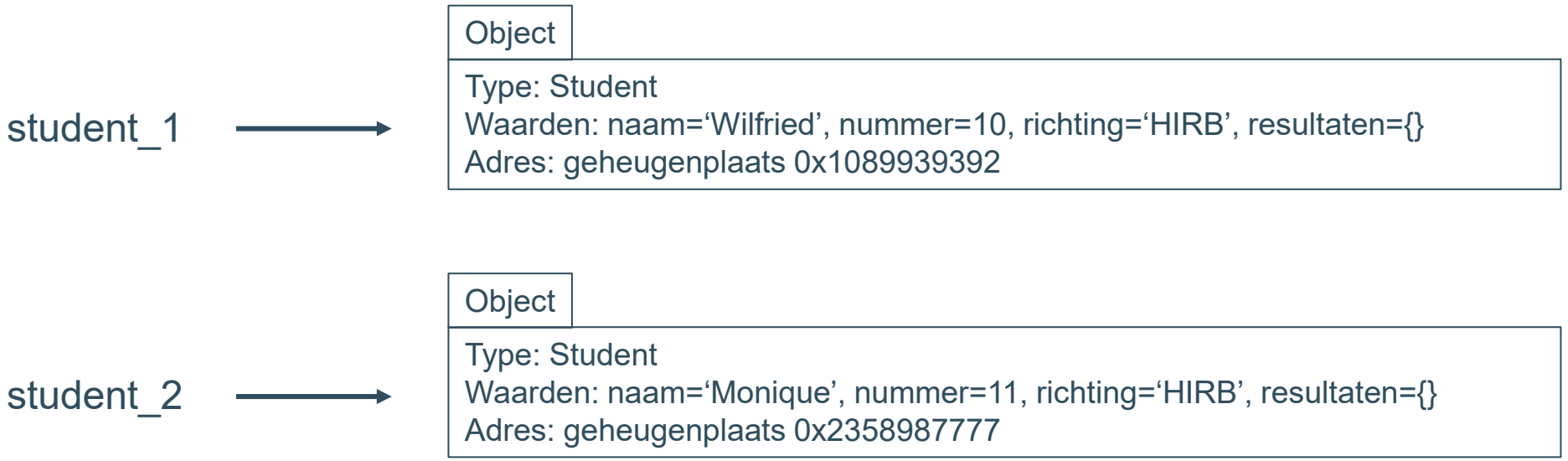
→ Nu kunnen we de attributen en methodes van de klasse oproepen over het object met het .-teken achter de variabele-naam

```
Student.naam
```

→ Dit kan niet. `Student` is een blauwdruk en een blauwdruk heeft geen specifieke naam, enkel de instanties van die blauwdruk

KU LEUVEN

Analogie met eerdere variabelen



KU LEUVEN

Methodes gebruiken

```
student_1.geef_punt_voor_vak('grondslagen', 10)
student_2.geef_punt_voor_vak('BIS', 18)
student_2.geef_punt_voor_vak('grondslagen', 4)

print('Resultaten student 1:', student_1.resultaten)
print('Resultaten student 2:', student_2.resultaten)
```

- We gebruiken de methodes om de `Student`-objecten punten te geven
- In de achtergrond worden hun aparte dictionaries ingevuld

Output:

```
Resultaten student 1: {'grondslagen': 10}
Resultaten student 2: {'BIS': 18, 'grondslagen': 4}
```

student_1	student_2
naam = ‘Wilfried’ nummer = 10 richting = ‘HIRB’ resultaten = {‘grondslagen’:10}	naam = ‘Monique’ nummer = 11 richting = ‘HIRB’ resultaten = {‘BIS’:18, ‘grondslagen’:4}

KU LEUVEN

Waarom nuttig?

- Doorgronden bepaalde syntax:
  - Bv. hoe werkt `mijn_string.lower()`
- Belangrijk voor modules:
  - Numpy:
    - Array-objecten om n-dimensionale matrices voor te stellen
  - Pandas:
    - Series-objecten om kolomdata te verwerken
    - DataFrame-objecten om tabulaire data te verwerken

Oefening 1.10: voorspel de output

```
class Teller:
    def __init__(self):
        self.n = 0

a = Teller()
b = a
b.n = 5
print(a.n, b.n, a is b)
```

1. **Voorspel eerst (30 s)**
2. **Dan runnen en verbeteren**

11. Modules

1A.14-modules.ipynb

KU LEUVEN

Modules importeren

- Python laat het toe om modules te importeren
- Bijvoorbeeld:
  - math: ingebouwde module voor wiskundige bewerkingen
  - time: ingebouwde module om met tijd te werken
  - NumPy: externe module voor wiskundige bewerkingen en matrix rekenen
  - scikit-learn: externe module voor data analyse
  - Pandas: externe module voor dataverwerking

Importeren

- Gebruik het `import` keyword + de naam
- Altijd opletten wat er in de modules zit
- Kan in PyCharm/Spyder/VSC door te hoveren

Code:

```
import math

for m in dir(math):
    print(m)
```

Code:

```
print('Pi:', math.pi)
print('cosinus(10):', math.cos(10))
```

Output:

```
Pi: 3.141592653589793
cosinus(10): -0.8390715290764524
```

Output:

```
__doc__
__file__
__loader__
__name__
__package__
__spec__
acos
acosh
asin
…
degrees
dist
e
erf
erfc
exp
expm1
fabs
factorial
floor
…
log
log10
…
nan
perm
pi
pow
prod
…
tanh
tau
trunc
```

functies

getallen

KU LEUVEN

Selectief importeren

- Gebruik `from` gevolgd door de module naam en `import` om aan te duiden welke onderdelen moeten geïmporteerd worden

Code:

```
from math import pi, cos
print('Type van pi:', type(pi))
print('Type van cos:', type(cos))
print('Cosinus(0):', cos(0))
```

Output:

```
Type van pi: <class 'float'>
Type van cos: <class 'builtin_function_or_method'>
Cosinus(0): 1.0
```


KU LEUVEN

De `time` module

- Handig om berekeningstijd van programma's te meten

Output:

```
import time

eerder = time.time()
print('Huidige tijd (aantal seconden sinds 01/01/1970):',
eerder)

# pauseer het programma 1 seconde
time.sleep(1)
later = time.time()
print('Duur van uitvoeren (s):', later - eerder)
```

Code:

```
Huidige tijd (aantal seconden sinds 01/01/1970):
1630397354.556272



Duur van uitvoeren (s): 1.0047059059143066
```

Alias

- Onderdelen van modules kunnen een andere naam gegeven worden
- Opletten: functies blijven functies!
- In data science bv.:
  - NumPy => np
  - Pandas => pd
  - TensorFlow => tf



https://www.reddit.com/r/ProgrammerHumor/comments/o3zjp4/thats_the_evilest_thing_i_can_imagine/

Module alias

Output:

```
import numpy as np

gem = np.mean([1, 2, 3])
pi = np.pi
print(round(gem * pi, 3))



from numpy import mean as gemiddelde

print(type(gemiddelde))
print('Gemiddelde van:', gemiddelde([10, 12, 14]))
```

Code:

```
6.283



<class 'function'>
Gemiddelde van: 12.0
```


KU LEUVEN

Oefening 1.11: voorspel de output

```
import math as m
print(m.sqrt(16))

m = 10
print(m.sqrt(4))

from math import ceil
print(ceil(3.2), floor(3.7))
```

Welke 2 regels in de notebook crashen — en met welke fout?

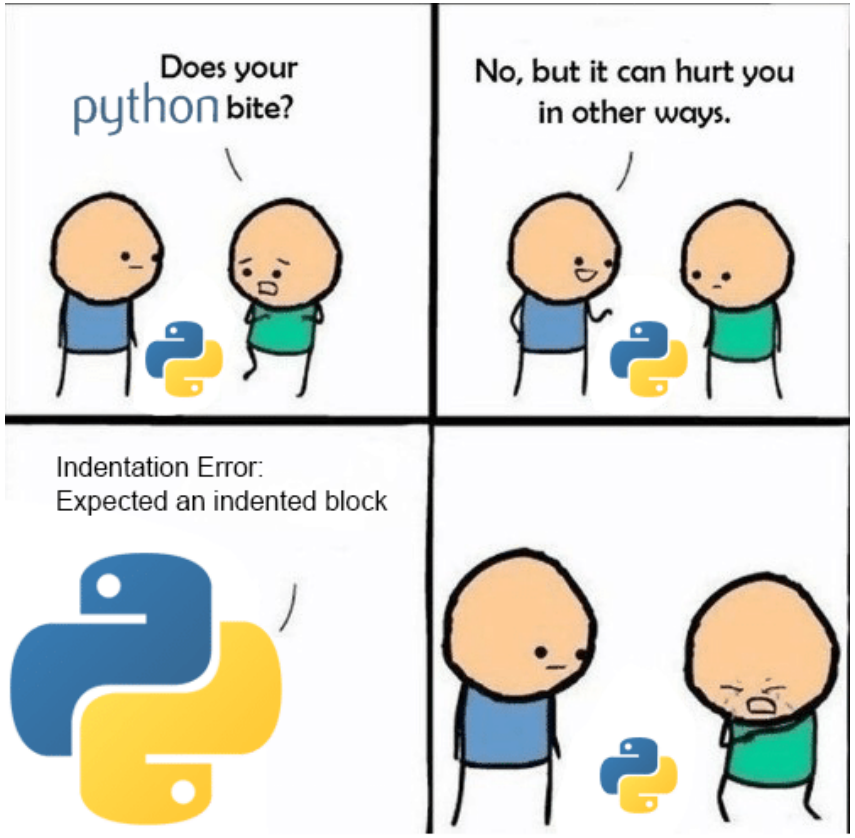
12. Fouten en debuggen

1A.15-fouten_debuggen.ipynb

KU LEUVEN

Introductie





He doesn't.

https://www.reddit.com/r/ProgrammerHumor/comments/j5nlpc/python_really_bites_you/
https://jovian.com/blog/tutorials/python-common-errors

Errors en exceptions

- **Errors** - fout in code (tekst):
  - Syntax errors: code is fout geschreven
  - Bv. spatie vergeten, komma op de verkeerde plaats, etc.
  - `SyntaxError, IndentationError`
- **Exceptions** - worden opgeroepen wanneer er iets inhoudelijk misgaat in de code:
  - Voornamelijk voor logische errors: code is syntactisch juist, maar inhoud klopt niet
  - Worden meestal expliciet ingewerkt in de code
  - Verschillende exceptions voor verschillende errors
  - Bv. `ZeroDivisionError`, `FileNotFoundError`, `NameError`

Veelvoorkomende syntaxfouten (1)

```
    if condition  # dubbelpunt mist
    print("Dit resulteert in een syntax error")
```

⊗ 0.0s

```
  Cell In[3], line 1
    if condition  # dubbelpunt mist
                  ^
SyntaxError: invalid syntax
```

```
    message = "Hallo, dit is een syntax error'
    print(message)
```

⊗ 0.0s

```
  Cell In[3], line 1
    message = "Hallo, dit is een syntax error'
                                              ^
SyntaxError: EOL while scanning string literal
```

```
    mijn_lijst = [1, 2, 3   # laatste rechte haak ontbreekt
    print(mijn_lijst[1])
```

⊗ 0.0s

```
  Cell In[7], line 2
    print(mijn_lijst[1])
    ^
SyntaxError: invalid syntax
```


KU LEUVEN

Veelvoorkomende syntaxfouten (2)

```
a = 5 b = 10  # er mist een komma tussen de statements
print(a, b)
```

⊗ 0.0s

```
  Cell In[5], line 1
    a = 5 b = 10  # er mist een komma tussen de statements
          ^
SyntaxError: invalid syntax
```

```
class = "MijnKlasse"  # "class" is een 'reserved' keyword
print(class)
```

⊗ 0.0s

```
  Cell In[4], line 1
    class = "MijnKlasse"  # "class" is een 'reserved' keyword
          ^
SyntaxError: invalid syntax
```

```
123variabele = "Invalide variabelenaam"  # variabelenaam kan niet met nummer beginnen
```

⊗ 0.0s

```
  Cell In[6], line 1
    123variabele = "Invalide variabelenaam"  # variabelenaam kan niet met nummer beginnen
       ^
SyntaxError: invalid syntax
```


KU LEUVEN

Veelvoorkomende fouten

- `IndentationError`:
  - Meer specifieke versie van een `SyntaxError`
  - Code is fout gestructureerd
  - Bv. een tab vergeten na een for-lus of if-statement

```
    for i in range (0, 10):
    for j in range(0, 10):
    print(i)
```

```
  Cell In[1], line 2
    for j in range(0, 10):
    ^
IndentationError: expected an indented block
```


KU LEUVEN

Veelvoorkomende fouten

- `IndentationError`:
  - Let op: je kan een tab ook qua logica op de verkeerde plaats zetten, Python zal niet ineens jouw code laten draaien zoals jij dat wenst

```
matrix = [[1,2,3], [3,4,5]]

for i in range(len(matrix)): # overloopt de rij indices
    for j in range(len(matrix[0])): # overloop de kolom indices (gegeven dat de lijsten in de lijst even lang zijn)
        print('i en j', i,j)
    print('Element op plaats i,j:', matrix[i][j])
```

✓ 0.0s

```
i en j 0 0
i en j 0 1
i en j 0 2
Element op plaats i,j: 3
i en j 1 0
i en j 1 1
i en j 1 2
Element op plaats i,j: 5
```

Deze code print niet elk element uit de matrix zoals misschien verwacht

KU LEUVEN

Veelvoorkomende exceptions

- `ZeroDivisionError`:
  - Bij delen door 0
  - Als je bv. een variabele gebruikt als noemer die 0 kan zijn

```
a = 5 / 0
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[8], line 1
----> 1 a = 5 / 0

ZeroDivisionError: division by zero
```

Veelvoorkomende exceptions

- `NameError`:
  - Als een variabele/functie wordt opgeroepen die (nog) niet bestaat

```
a = 3
b = 4
print(a + c)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[13], line 3
      1 a = 3
      2 b = 4
----> 3 print(a + c)

NameError: name 'c' is not defined
```

`print()` verkeerd geschreven

```
pint('Johannes')
```

⊗ 0.0s

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/Users/johannesdesmedt/Desktop/Teaching/SOEP1/Deel 1/1.12-errors_0910.ipynb Cell 15 line 1
----> 1 pint('Johannes')

NameError: name 'pint' is not defined
```


KU LEUVEN

Veelvoorkomende exceptions

- `ValueError`:
  - De waarde die wordt meegegeven kan verwerkt worden (type is juist), maar past niet binnen de berekening die erover gebeurt

```
import math
a = 10
b = -10
c = math.sqrt(a)
d = math.sqrt(b)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[9], line 5
      3 b = -10
      4 c = math.sqrt(a)
----> 5 d = math.sqrt(b)

ValueError: math domain error
```

-10 is een integer/getal en dus een valide argument qua *type*, maar vierkantswortel van -10 bestaat niet (reëel) – de *waarde* (value) is fout

KU LEUVEN

Veelvoorkomende exceptions

- `ValueError`:

```
    values = []
    max_value = max(values)
    min_value = min(values)

⊗ 0.0s
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/Users/johannesdesmedt/Desktop/Teaching/SOEP1/Deel 1/1.12-errors_0910.ipynb Cell 18 line 2
      1 values = []
----> 2 max_value = max(values)
      3 min_value = min(values)

ValueError: max() arg is an empty sequence
```

`max()/min()` kunnen gebruikt worden over een lijst, maar geen lege

KU LEUVEN

Veelvoorkomende exceptions

- `ValueError`:

```
for a, b in [(1, 2, 3), (3, 4, 5)]:
    print(a, b)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[11], line 1
----> 1 for a, b in [(1, 2, 3), (3, 4, 5)]:
      2     print(a, b)

ValueError: too many values to unpack (expected 2)
```

Er zitten 3 elementen in de elementen van de lijst (een tuple van 3), maar er staan maar 2 variabelen (`a` en `b`) klaar in de for-lus

```
for a, b, c in [(1,2), (3,4)]:
    print(a)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 for a, b, c in [(1,2), (3,4)]:
      2     print(a)

ValueError: not enough values to unpack (expected 3, got 2)
```

Er zitten 2 elementen in de elementen van de lijst (een tuple van 2), maar er staan 3 variabelen (`a`, `b` en `c`) klaar

KU LEUVEN

Veelvoorkomende exceptions

- `KeyError`:
  - Wanneer een bepaalde sleutel in een dictionary wordt opgevraagd die niet bestaat
  - Wanneer een waarde uit een set wordt verwijderd die niet bestaat

```
a = {'a': 1, 'b':2}
print(a['a'])
print(a['c'])
```

```
1

---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[12], line 3
      1 a = {'a': 1, 'b':2}
      2 print(a['a'])
----> 3 print(a['c'])

KeyError: 'c'
```


KU LEUVEN

Veelvoorkomende exceptions

- `IndexError`:
  - Een index uit een lijst wordt opgevraagd die niet bestaat (bv. omdat je te ver geteld hebt)

```
lijst = [0, 1, 2, 3]
for i in range(0, 5):
    print(lijst[i])
```

```
0
1
2
3
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[14], line 3
      1 lijst = [0, 1, 2, 3]
      2 for i in range(0, 5):
----> 3     print(lijst[i])

IndexError: list index out of range
```

Slimmer (werkt voor elke lijst):

```
lijst = [0, 1, 2, 3]
for i in range(0, len(lijst)): # of zelfs gewoon range(len(lijst))
    print(lijst[i])
```

✓ 0.0s

```
0
1
2
3
```


KU LEUVEN

Veelvoorkomende exceptions

- `AttributeError`:
  - Attributen van een object (bv. een een lijst of een student) bestaan uit enkele attributen (bv. leeftijd) en de bijbehorende methodes (bv. `append()` of `geef_naam()`)
  - Opvragen van attributen die niet bestaan voor het object type (bv. `lower()` bij een lijst of `append()` bij een set) resulteert in een `AttributeError`

```
a = 'Johannes'
print(a.lower())
a = 10
print(a.lower())
```

```
johannes

---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[15], line 4
      2 print(a.lower())
      3 a = 10
----> 4 print(a.lower())

AttributeError: 'int' object has no attribute 'lower'
```


KU LEUVEN

Veelvoorkomende exceptions

- `AttributeError`:

```
een_lijst = [10, 20]
een_set = set()

print(een_lijst.append(10))
print(een_set.append(10))
```

⊗ 0.0s

```
None

---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/Users/johannesdesmedt/Desktop/Teaching/SOEP1/Deel 1/1.12-errors_0910.ipynb Cell 30 line 5
      2 een_set = set()
      4 print(een_lijst.append(10))
----> 5 print(een_set.append(10))

AttributeError: 'set' object has no attribute 'append'
```


KU LEUVEN

Veelvoorkomende exceptions

- `TypeError`:
  - Een functie of operatie wordt opgeroepen over een verkeerd type

```
a = 'johannes' + 2
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[17], line 1
----> 1 a = 'johannes' + 2

TypeError: can only concatenate str (not "int") to str
```


KU LEUVEN

Veelvoorkomende exceptions

- `TypeError`:

```
a = [1, 2, 3]
b = a - [1]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/Users/johannesdesmedt/Desktop/Teaching/SOEP1/Deel 1/1.12-errors_2509.ipynb Cell 25 line 2
      1 a = [1, 2, 3]
----> 2 b = a - [1]

TypeError: unsupported operand type(s) for -: 'list' and 'list'
```


KU LEUVEN

Veelvoorkomende exceptions

- `OverflowError:`
  - De numerische operatie gaat buiten wat het data type ondersteunt (vooral bij kommagetallen)

```
j = 10.0
for i in range(1, 10000):
    j = j ** i
    print(j)
```

```
10.0
100.0
1000000.0
1e+24
1e+120

---------------------------------------------------------------------------
OverflowError                             Traceback (most recent call last)
Cell In[11], line 3
      1 j = 10.0
      2 for i in range(1, 10000):
----> 3     j = j ** i
      4     print(j)
      5     # print(str(j).format('{.3e}'))

OverflowError: (34, 'Result too large')
```


KU LEUVEN

Veelvoorkomende exceptions

- `MemoryError`:
  - Je gebruikt te veel geheugen
  - Dit gebeurt veel op het examen bij eindeloze loops:
    - Je herkent dit door de [*] naast je cell -> cell blijft uitvoeren, meestal blijf je dan ook extra geheugen opvragen
    - Herstart dan de kernel!

```
gigantische_lijst = []
for i in range(0,10**1000):
    for j in range(0, 10**100):
        for k in range(0, 10*100):
            gigantische_lijst.append('Een kleine string'*1000)
```

```
Cannot execute code, session has been disposed. Please try restarting the Kernel.

The Kernel crashed while executing code in the the current cell or a previous cell.
```

Opvangen van een exception

- Met `try` en `except` kan je gericht fouten opvangen:
  - Wanneer je weet dat een fout kan optreden en je een omweg wil voorzien
  - Zoals de voorbeelden in dit slide deck waarbij we foute code illustreren

```
try:
    a = 10 / 0
except ZeroDivisionError as e:  # syntax error, er mist een dubbele punt
    print(e)
    print("Je kan niet delen door nul")
```

We vangen hier een specifieke fout op en printen ze als een nieuwe variabele `e`

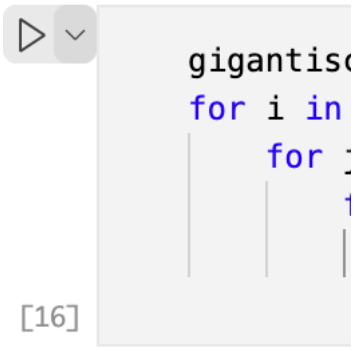
```
try:
    a = [0, 1, 2]
    print(a[3])  # IndexError: list index out of range
except IndexError as e:
    print("Indexfout: de opgegeven index bestaat niet")
except ZeroDivisionError as e:
    print("Nuldelingfout: je kan niet delen door nul")
```

We kunnen zo meerdere `excepts` toevoegen om gericht feedback te verschaffen

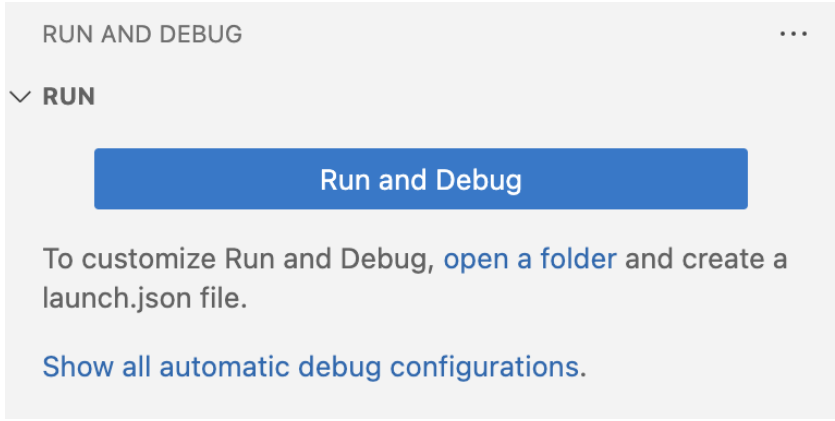
KU LEUVEN

Debuggen

- In Jupyter notebook:
  - Geen debugger
  - `print()`-statements helpen je op weg
- In Visual Studio Code:
  - Ingebouwde debugger



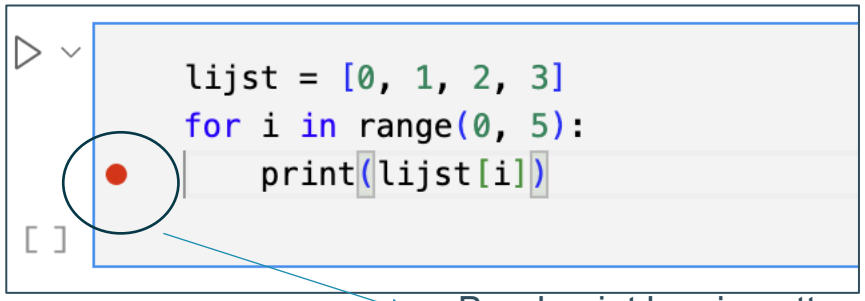
Gebruik het pijltje en kies ‘Debug cell’



Klik op ‘Run and Debug’

Debuggen (VSC)

- Tijdens het debuggen kunnen we 1 of meerdere ‘breakpoints’ gebruiken
- VSC:



Breakpoint kan je zetten door naast de code te klikken



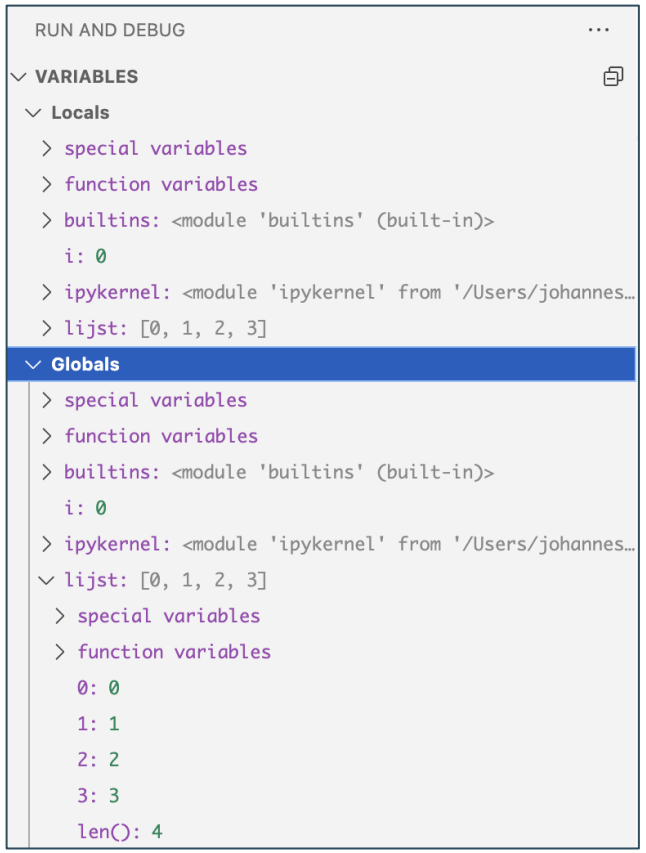
Runt cell terug opnieuw

Begint de cell te runnen tot het breakpoint

Gaat 1 lijn in de code verder

Zijn bedoeld om ‘in’ en ‘uit’ een functie te springen als je die tegenkomt in de code (maar dus bv. ook de low-level code van `print()`)

In het debug-menu kan je alle variabelen opvolgen:



KU LEUVEN

Oefening 1.12: voorspel de fout

```
l = [1, 2, 3]
totaal = 0
for i in range(len(l) + 1):
    totaal += l[i]
print(totaal)

leeftijd = '20'
print(leeftijd + 1)
```

Welk fouttype — en bij welke i / welke regel?

KU LEUVEN